Design Proposal

Sprint-Based Planning for `mcprojsim`

Version: 1.2.0

Johan Persson
<johan162@gmail.com>

Generated on April 19, 2026

Contents

1 Sprint-Based Planning **3**

1.1 Executive Summary . . . 3
1.2 Query Type . . . 4
1.3 Current `mcprojsim` Architecture and Why It Matters . . . 4
1.4 Architecture Overview . . . 5
1.5 Key Terms Used in This Proposal . . . 5
1.6 External Research: What Agile Practice Suggests . . . 5
1.7 Compared Approaches . . . 6
1.8 Recommended Product Design . . . 7
1.8.1 1. Add sprint planning as a distinct planning mode . . . 7
1.8.2 Alternative external history source . . . 9
1.8.3 Example complete project file . . . 10
1.8.4 Example project file using external history . . . 13
1.8.5 External history file formats . . . 14
1.8.6 Historical sprint data fields . . . 14
1.8.7 What `holiday_factor` means . . . 17
1.8.8 Purpose of `holiday_factor` . . . 17
1.8.9 How `holiday_factor` should affect planning and prediction . . . 18
1.8.10 Support two planning units, but keep them separate . . . 19
1.8.11 Use a dependency-aware sprint pull simulator . . . 19
1.8.12 Make empirical resampling the default uncertainty model . . . 20
1.8.13 Treat spillover, scope-addition, and scope-removal as first-class historical signals 21
1.8.14 Add an explicit volatility layer, but keep it optional . . . 23
1.8.15 Project-file parameter clarifications . . . 27
1.9 Measuring Uncertainty and Volatility . . . 31
1.10 How to Handle Adjustable Sprint Lengths . . . 31
1.11 How to Treat Partial Work and Carryover . . . 32
1.12 Data Model Recommendation . . . 33
1.12.1 New project-file structures . . . 33
1.12.2 Task-level additions . . . 35
1.13 Recommended Internal Component Design . . . 36

1.14 Integration with Existing `mcprojsim` Concepts . . . 36
1.14.1 Reuse what already fits . . . 36
1.14.2 Do **not** force-fit what does not . . . 37
1.15 MVP vs. Follow-On Roadmap . . . 37
1.15.1 MVP . . . 37
1.15.2 Phase 2 . . . 37
1.15.3 Phase 3 . . . 38
1.16 Implementation Notes . . . 38
1.17 Why This Is the Best-Fit Approach for `mcprojsim` . . . 40
1.18 Confidence Assessment . . . 40
1.19 Footnotes . . . 41

2 Statistical Methods and Approach 42
2.1 Canonical Historical Observation . . . 42
2.2 Delivery-Side Normalization . . . 43
2.3 Sprint-Entity Resampling and Mixed Sprint Lengths . . . 43
2.4 Joint Empirical Bootstrap . . . 44
2.5 Derived Historical Ratios . . . 45
2.6 Planned-Load Guidance Heuristic . . . 45
2.7 Backlog-State Recursion . . . 45
2.8 Volatility Overlay . . . 46
2.9 Task-Level Execution Spillover . . . 46
2.10 Output Statistics . . . 47
2.11 Why These Methods Fit `mcprojsim` . . . 47

3 Natural Language Input and MCP Integration 48
3.1 Suggested Natural-Language Shape . . . 48
3.2 Accepted Human Variants . . . 49
3.3 Natural-Language Example . . . 50
3.4 Scope of the Natural-Language Form . . . 52

4 Formal Requirements 53
4.1 Configuration Requirements . . . 53
4.1.1 FR-SP-001: Sprint Planning Mode Integration . . . 53
4.1.2 FR-SP-002: Sprint Length Configuration . . . 53
4.1.3 FR-SP-003: Capacity Planning Units . . . 53
4.1.4 FR-SP-004: Historical Sprint Outcome Input . . . 54
4.1.5 FR-SP-005: Sprint Planning Defaults . . . 55
4.1.6 FR-SP-006: Empirical Joint Outcome Resampling . . . 55
4.1.7 FR-SP-007: Historical Spillover Modeling . . . 56
4.1.8 FR-SP-008: Historical Scope-Addition Modeling . . . 56
4.1.9 FR-SP-009: Historical Scope-Removal Modeling . . . 56
4.1.10 FR-SP-010: Historical Metadata Treatment . . . 56

4.2 Simulation Behavior Requirements . . . 57
4.2.1 FR-SP-011: Dependency-Aware Sprint Pull Simulation . . . 57
4.2.2 FR-SP-012: Future Sprint Override Configuration . . . 57
4.2.3 FR-SP-013: Volatility Overlay . . . 58
4.2.4 FR-SP-014: Task-Level Execution Spillover Modeling . . . 58
4.2.5 FR-SP-015: Spillover Effort Carryover . . . 59
4.3 Output and Reporting Requirements . . . 59
4.3.1 FR-SP-016: Sprint Simulation Output and Statistics . . . 59
4.3.2 FR-SP-017: Carryover and Spillover Diagnostics . . . 60
4.3.3 FR-SP-018: Export and Reporting . . . 60
4.4 Validation Requirements . . . 61
4.4.1 FR-SP-019: Schema and Validation . . . 61
4.5 Implementation Structure Requirements . . . 62
4.5.1 FR-SP-020: Internal Component Structure . . . 62

5 Implementation Work Breakdown 63
5.1 Extend the Project Schema First . . . 63
5.2 Add Raw Validation and Source-Aware Error Reporting . . . 64
5.3 Implement the New Planning Engine as a Parallel Subsystem . . . 64
5.4 Add a Dedicated Sprint Results Model . . . 65
5.5 Integrate the CLI Without Breaking the Current Command Model . . . 66
5.6 Extend Exporters with a Parallel Sprint-Planning Section . . . 66
5.7 Add Tests in the Same Layers the Repository Already Uses . . . 67
5.8 Recommended Implementation Order . . . 67

6 MVP-Only Implementation Plan 69
6.1 MVP Step 1. Schema and Validation . . . 69
6.2 MVP Step 2. Historical Normalization and Resampling . . . 70
6.3 MVP Step 3. Dependency-Aware Sprint Planner . . . 71
6.4 MVP Step 4. Sprint Simulation Engine and Results Model . . . 71
6.5 MVP Step 5. CLI and Export Integration . . . 72
6.6 MVP Step 6. Tests . . . 72

7 Natural-Language Parser Extension Plan 74
7.1 Main parser functions to change . . . 74
7.1.1 1. Extend the parsed data model . . . 74
7.1.2 2. Extend `parse()` with sprint-planning sections . . . 74
7.1.3 3. Add sprint-planning metadata helpers . . . 75
7.1.4 4. Add normalization helpers for units and aliases . . . 75
7.1.5 5. Extend `to_yaml()` to emit `sprint_planning` . . . 76
7.1.6 6. Keep `parse_and_generate()` unchanged in shape . . . 76
7.2 Suggested implementation order inside `nl_parser.py` . . . 76
7.3 Verification targets for the natural-language path . . . 76

8 Some considerations for using an MCMC model **77**
8.1 The Generative Model . . . 77
8.2 The Statistical Model . . . 77
8.3 MCMC Implementation Strategy . . . 79
8.4 Prediction: Rolling Forward . . . 79
8.5 Model Extensions . . . 80
8.5.1 Extension Implementation . . . 80
8.5.2 Validation Strategy . . . 80
8.6 Key Advantage of This Approach . . . 80

9 Alternative Approache using Negative Binomial Distribution **81**
9.1 How to find the model parameters from the historical data? . . . 81
9.1.1 Using MLE to estimate the parameters . . . 81
9.1.2 Using Bayesian inference to estimate the parameters . . . 82

10 Addendum: Team-Size-Aware Sprint Forecasting **83**
10.1 Why Team Size Should Be Modeled Explicitly . . . 83
10.2 Statistical Design: Separate Team Capacity From Team Effectiveness . . . 83
10.2.1 Step 1: Normalize to per-person baseline . . . 84
10.2.2 Step 2: Empirical joint resampling on normalized vectors . . . 84
10.2.3 Step 3: Scale sampled outcomes by future sprint staffing . . . 84
10.3 Non-Linear Team-Size Effects (Optional, Recommended) . . . 84
10.4 Schema Additions . . . 85
10.4.1 New default staffing setting . . . 85
10.4.2 Extend `future_sprint_overrides` . . . 85
10.5 Updated Canonical Models . . . 86
10.5.1 `SprintPlanningSpec` additions . . . 86
10.5.2 `FutureSprintOverrideSpec` additions . . . 86
10.5.3 `SprintHistoryEntry` expectations . . . 86
10.6 Validation Requirements for Team Size . . . 86
10.7 Simulation Algorithm Changes . . . 86
10.8 Reporting and Diagnostics Additions . . . 87
10.9 Design for Future Sprint Staffing Input . . . 87
10.10 Example: Complete Team-Size-Aware Sprint Planning Block . . . 88
10.11 Implementation Plan in This Repository . . . 88
10.11.1 1. Schema layer . . . 88
10.11.2 2. Parser and validation layer . . . 89
10.11.3 3. Sprint sampler and engine layer . . . 89
10.11.4 4. Results and exporter layer . . . 89
10.11.5 5. CLI and MCP layer . . . 89
10.12 Backward Compatibility and Migration . . . 90
10.13 Best-Use Recommendation . . . 90

Version: 1.2.0

Chapter 1

Sprint-Based Planning

1.1 Executive Summary

`mcprojsim` already produces two important distributions per simulation run: elapsed project duration and total effort. It does that by sampling task uncertainty, applying multiplicative uncertainty factors and additive risks, then scheduling the resulting task durations through dependency-only or resource-constrained scheduling. A sprint-planning mode should therefore be added **alongside** the current engine, not as a replacement for it.

The most suitable approach is an **empirical Monte Carlo sprint forecast** driven by historical team sprint outcomes, with two capacity modes: **story points per sprint** and **tasks/items per sprint**. Story-point mode is appropriate when the team already plans and measures in points; task-throughput mode is appropriate only when work items are right-sized and reasonably homogeneous, which is exactly the condition emphasized in throughput-driven sprint planning guidance.

The best way is to make a **dependency-aware sprint simulator** that repeatedly samples sprint capacity from historical data, but with each historical sprint treated as a joint outcome containing **completed work, spillover work, mid-sprint added work, and mid-sprint removed work**. The simulator should pull a subset of ready tasks into each sprint, model scope added during the sprint, model work explicitly removed from the sprint after planning, carry unfinished work out of the sprint, and stop when all project tasks are complete. The output should be a distribution of **sprints-to-done** (P50/P80/P90), plus date projections, burn-up style percentile bands, commitment guidance for how much planned work to load into future sprints, and volatility diagnostics such as standard deviation and coefficient of variation, reusing the same style of summary statistics already used elsewhere in the product.

The strongest design choice is to make the sprint-capacity model **empirical first** (bootstrap/resampling historical sprint outcomes) rather than parametric first (fit a Normal/Lognormal model to velocity). Scrum and agile forecasting guidance consistently emphasizes using observed variation instead of collapsing data to a single average, and the current product direction already aligns well with Monte Carlo-style sampling of uncertain work.

1.2 Query Type

This is a **technical deep-dive / architecture proposal**: it asks how to add a new sprint-based forecasting mode to an existing Monte Carlo project simulation system, how to ground it in agile delivery practice, and how to represent uncertainty and volatility in a statistically sound way.

1.3 Current `mcprojsim` Architecture and Why It Matters

The current simulation engine runs many iterations and, in each one, resolves symbolic estimates, samples a task duration distribution, applies multiplicative uncertainty factors, applies risk impacts, and then schedules the resulting task durations to produce project duration statistics. This means the system is already designed around **repeated stochastic simulation**, so sprint planning should plug into the same style of computation rather than introducing a deterministic planning subsystem.

The engine also stores a per-iteration **effort distribution** by summing all task durations, separately from elapsed duration. That is important because sprint planning is conceptually closer to **capacity vs. backlog consumption** than to critical-path elapsed time; the product already distinguishes those two ideas.

`SimulationResults` already exposes mean, median, standard deviation, skewness, kurtosis, percentile lookup, and dictionary/export support for simulated outputs. That suggests the cleanest extension is to add a parallel result surface for sprint forecasts, rather than trying to coerce sprint metrics into the existing `durations` array.

The staffing analyzer further reinforces that today's model is fundamentally **effort + capacity => calendar time**, with team-size effects and communication overhead applied after simulation. Sprint planning is a different abstraction: it assumes a fixed sprint cadence and an empirically observed team delivery capacity per sprint, then asks how many sprint buckets are required to finish a backlog.

Finally, the current program already models dependencies, resource availability, holidays, planned absences, sickness, and practical caps on parallelism. A sprint-planning mode should respect task dependencies and can optionally adjust sampled sprint capacity for known future calendar effects.

1.4 Architecture Overview

```
Current model
-------------
Task estimates -> sample task durations -> apply uncertainty/risks
               -> schedule tasks by deps/resources/calendars
               -> elapsed-duration distribution + effort distribution

Recommended sprint model
------------------------
Project tasks + sprint config + historical sprint outcomes
             -> build ready queue from dependencies
             -> sample {completed, spillover, added, removed} profile for Sprint 1
             -> pull ready tasks/items into Sprint 1
             -> inject unplanned added work, remove de-scoped work, and carry over unfinished work
             -> update dependencies / remaining backlog
             -> repeat until all tasks complete
             -> sprint-count distribution + sprint-date distribution
```

1.5 Key Terms Used in This Proposal

- **Effort distribution**: the existing `mcprojsim` distribution produced by summing sampled task durations in time units such as hours or days.
- **Backlog consumption**: the sprint-planning view of progress, measured as completed planning units per sprint such as story points or tasks.
- **Historical spillover**: unfinished planned work recorded in historical sprint outcome rows through the `spillover_*` fields.
- **Execution spillover**: task-level overrun during simulation, where a task is started, consumes capacity, and still misses sprint end.
- **Ready queue**: the set of project tasks whose declared dependencies have all been completed and are therefore eligible to be pulled into the next sprint.

1.6 External Research: What Agile Practice Suggests

The Scrum Guide defines Sprints as fixed-length events of one month or less, and the entire framework is explicitly empirical: teams inspect outcomes and adapt based on what actually happened.[1] That supports a sprint-planning feature whose primary inputs are **historical sprint outcomes** and a **fixed sprint length**.

Atlassian's velocity guidance describes sprint velocity as the amount of work a Scrum team completes in a sprint, usually in story points, and it explicitly says velocity should be based on **fully completed stories**, averaged across multiple sprints, while also noting that team size, experience, story complexity, and holidays affect it and that velocity is team-specific.[2] That validates story-

[1] `https://scrumguides.org/scrum-guide.html` (The Scrum Guide, section "The Sprint")
[2] `https://www.atlassian.com/agile/project-management/velocity-scrum` (Atlassian, "Velocity in Scrum")

point-based sprint capacity as one supported mode, but it also highlights a key limitation: a simple average is not enough when capacity varies materially from sprint to sprint, when items spill out of the sprint, or when urgent work is added after sprint start.

Scrum.org's Monte Carlo forecasting article makes the crucial statistical point: using a single average burn rate discards variation, while Monte Carlo forecasting keeps the observed spread and yields a **range** of likely outcomes rather than a falsely precise point forecast.[3] That is directly aligned with `mcprojsim`'s Monte Carlo philosophy and is the best argument against implementing sprint planning as "remaining backlog / average velocity".

Scrum.org's throughput-driven sprint planning article argues for using **throughput** (completed items per unit time) rather than story points when teams manage similarly sized work items, and it ties that to a Service Level Expectation (SLE) that helps determine whether items are "right-sized" for the workflow.[4] This is the best support for a second capacity mode based on **tasks/items per sprint**, but it also implies a design constraint: item-throughput forecasting only works well when tasks are small enough and similarly sized enough to behave like comparable work items.

The LeadingEDJE agile forecasting write-up reinforces the same pattern: flow metrics and Monte Carlo simulation should be used to answer either "how many items by date?" or "when will this amount of work finish?", using historical throughput or cycle-time data instead of deterministic plans.[5] That maps almost exactly to the user request for "distribution of how many sprints it takes to complete the total effort to certain percentile."

1.7 Compared Approaches

Approach	How it works	Strengths	Weaknesses	Verdict
Average velocity / average throughput	Divide remaining backlog by mean points-per-sprint or tasks-per-sprint	Very simple, easy to explain	Throws away variation; produces brittle point forecasts; weak under volatility	Do **not** use as primary forecast

[3] `https://www.scrum.org/resources/blog/monte-carlo-forecasting-scrum` (Scrum.org, "Monte Carlo forecasting in Scrum")

[4] `https://www.scrum.org/resources/blog/throughput-driven-sprint-planning` (Scrum.org, "Throughput-Driven Sprint Planning")

[5] `https://blog.leadingedje.com/post/agileforecasting/caseforit.html` (LeadingEDJE, "Agile Forecasting: Monte Carlo Simulations and Flow Metrics")

Approach	How it works	Strengths	Weaknesses	Verdict
Parametric capacity distribution	Fit a Normal/ Lognormal /Gamma/ Negative-Binomial model to capacity, then sample	Compact, smooth, supports extrapolation	Risk of fitting the wrong shape; fragile with small sample sizes; harder to explain to users	Optional advanced mode
Empirical Monte Carlo resampling	Sample future sprint outcomes from observed historical sprint outcomes	Preserves real observed variation and churn coupling; easy to explain; aligned with the product's Monte Carlo direction	Needs enough history; can inherit historical regime bias	**Recommended default**
Throughput + SLE flow forecasting	Forecast completed items using throughput and right-sized items	Works well for item flow; no story points required	Only valid if items are small and comparable; weaker for uneven task sizes	Recommended for task-count mode
Dependency-aware sprint simulation	Resample capacity, but also only pull dependency-ready tasks into each sprint	Matches project-task structure; yields subset-of-tasks-per-sprint plan	More implementation work; needs priority policy	**Recommended core simulator**

1.8 Recommended Product Design

1.8.1 1. Add sprint planning as a distinct planning mode

Add a new top-level project section such as `sprint_planning`, rather than overloading the current `project` or `simulation` sections. The existing engine is duration-based and the new feature is capacity/burn-up based, so the separation should be explicit in the schema and outputs.

Suggested shape:

```yaml
project:
  name: "Example"
  start_date: "2026-04-06"

sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: "story_points"   # or "tasks"
  removed_work_treatment: "churn_only"   # or "reduce_backlog"
  future_sprint_overrides:
    - sprint_number: 4
      holiday_factor: 0.8
      notes: "Public holiday sprint"
  history:
    - sprint_id: "SPR-001"
      sprint_length_weeks: 2
      completed_story_points: 23
      spillover_story_points: 5
      added_story_points: 3
      removed_story_points: 2
    - sprint_id: "SPR-002"
      sprint_length_weeks: 2
      completed_story_points: 19
      spillover_story_points: 8
      added_story_points: 6
      removed_story_points: 1
    - sprint_id: "SPR-003"
      sprint_length_weeks: 2
      completed_story_points: 26
      spillover_story_points: 2
      added_story_points: 1
      removed_story_points: 0
  uncertainty_mode: "empirical"
  volatility_overlay:
    enabled: true
    disruption_probability: 0.10
    disruption_multiplier_low: 0.5
    disruption_multiplier_expected: 0.8
    disruption_multiplier_high: 1.0
```

The system should support sprint length adjustable in whole weeks and capacity configurable either as tasks-per-sprint or story-points-per-sprint, so those need to be first-class config choices.

The extended historical fields for sprint churn should be optional in the input schema so teams can start with only completed work and then progressively add richer data. When omitted, `spillover_*`, `added_*`, and `removed_*` should default to `0` for that history row. This preserves backward compatibility for partially observed historical data while still rewarding teams that capture more detailed sprint outcomes.

Each historical sprint row should be identified by a mandatory `sprint_id` field. `sprint_id` is the stable key for storing, validating, and reporting a specific historical sprint record. By contrast, `end_date` should become optional metadata that can be used for reporting, chronology hints, or later segmentation, but it should no longer be the primary identifier for the row.

Each historical sprint row should also declare its delivery unit family by specifying exactly one of `completed_story_points` or `completed_tasks`. That completed field anchors the interpretation of the rest of the row. If the row uses `completed_story_points`, then any spillover, added, and removed values for that row should be expressed using the corresponding `*_story_points` fields. If the row uses `completed_tasks`, then the rest of the row should use the corresponding `*_tasks` fields. Mixed-unit rows should not be allowed, because they make the historical outcome vector ambiguous.

More generally, every historical sprint field other than `sprint_id` should be omittable and should fall back to a neutral, non-impacting default. This makes it possible to use partially observed historical data without forcing teams to backfill every sprint attribute before they can benefit from the model.

An additional post-MVP input mode should also be supported for teams whose sprint history is generated automatically from another system. In practice, that history is more likely to be exported from Jira, Azure DevOps, or an internal reporting script as CSV or JSON than manually copied into the main YAML or TOML project file. The design should therefore allow `sprint_planning.history` to be provided either inline as it is shown above or as a reference to an external history file in a supported machine-generated format.

The cleanest schema for that alternative is to allow `history` to accept exactly one of these two shapes:

- an inline list of history rows;
- an external source descriptor containing a file `path` and explicit `format` of `json` or `csv`.

This keeps the primary project file readable while letting teams refresh sprint history automatically from an external tool without rewriting the full YAML or TOML project definition.

Future calendar adjustments should also be explicitly configurable rather than only described conceptually. The cleanest approach is to allow a `future_sprint_overrides` list in `SprintPlanningSpec`, where each override targets a known future sprint and applies a `holiday_factor` or equivalent capacity multiplier for that specific sprint. This closes the gap between historical interpretation and forward prediction.

1.8.2 Alternative external history source

For teams that generate history automatically, `sprint_planning.history` should also allow an external source descriptor instead of inline rows.

Suggested YAML shape:

```yaml
sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: "story_points"
  history:
    format: "csv"   # or "json"
    path: "data/sprint_history.csv"
```

Design rules:

- exactly one history source mechanism should be allowed: either inline rows or an external file reference;
- the external file path should be resolved relative to the project file unless given as an absolute path;
- the external file should be loaded before default filling, normalization, and validation;
- once loaded, external rows should be treated identically to inline rows.

1.8.3 Example complete project file

The following example shows a project file that includes all currently proposed historical sprint data points:

```yaml
project:
  name: "Payments Platform Refresh"
  start_date: "2026-04-06"

simulation:
  iterations: 10000
  random_seed: 42

sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: "story_points"
  planning_confidence_level: 0.80
  removed_work_treatment: "churn_only"  # removed work informs churn, but does not shrink backlog
  uncertainty_mode: "empirical"
  future_sprint_overrides:
    - sprint_number: 3
      holiday_factor: 0.8
      notes: "Spring public holiday"
  history:
    - sprint_id: "SPR-2026-01"
      end_date: "2026-01-16"
      sprint_length_weeks: 2
      completed_story_points: 21
      spillover_story_points: 5
      added_story_points: 3
      removed_story_points: 1
      team_size: 5
      holiday_factor: 1.0
      notes: "Normal sprint"
    - sprint_id: "SPR-2026-02"
      end_date: "2026-01-30"
      sprint_length_weeks: 2
      completed_story_points: 18
      spillover_story_points: 7
      added_story_points: 6
      removed_story_points: 2
      team_size: 5
      holiday_factor: 0.9
      notes: "Production incident interrupted planned work"
    - sprint_id: "SPR-2026-03"
      end_date: "2026-02-13"
      sprint_length_weeks: 2
      completed_story_points: 24
      spillover_story_points: 2
      added_story_points: 1
      removed_story_points: 0
      team_size: 5
      holiday_factor: 1.0
      notes: "Low churn sprint"
```

```yaml
  volatility_overlay:
    enabled: true
    disruption_probability: 0.10
    disruption_multiplier_low: 0.5
    disruption_multiplier_expected: 0.8
    disruption_multiplier_high: 1.0

  spillover:
    enabled: true
    model: "table"
    size_reference_points: 5
    size_brackets:
      - max_points: 2
        probability: 0.05
      - max_points: 5
        probability: 0.12
      - max_points: 8
        probability: 0.25
      - max_points: null
        probability: 0.40
    consumed_fraction_alpha: 3.25
    consumed_fraction_beta: 1.75

tasks:
  - id: "discovery"
    name: "Discovery and architecture"
    planning_story_points: 5

  - id: "schema"
    name: "Schema updates"
    planning_story_points: 8
    dependencies: ["discovery"]

  - id: "api"
    name: "API implementation"
    planning_story_points: 8
    dependencies: ["schema"]

  - id: "ui"
    name: "UI integration"
    planning_story_points: 5
    dependencies: ["api"]

  - id: "rollout"
    name: "Rollout and validation"
    planning_story_points: 3
    dependencies: ["ui"]
```

Reading the example:

- `removed_work_treatment: "churn_only"` means removed work is used to measure planning churn and adjust commitment guidance, but does not reduce the remaining backlog forecast.
- `future_sprint_overrides` applies known future calendar adjustments such as public holidays to specific future sprints.
- `volatility_overlay` is an optional sprint-level disruption model layered on top of historical resampling.
- `spillover` configures the optional execution-spillover model used when a pulled task overruns within a sprint.

1.8.4 Example project file using external history

The following example shows the same idea using an external CSV file instead of inline history rows:

```
project:
  name: "Payments Platform Refresh"
  start_date: "2026-04-06"

simulation:
  iterations: 10000
  random_seed: 42

sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: "story_points"
  planning_confidence_level: 0.80
  removed_work_treatment: "churn_only"
  uncertainty_mode: "empirical"
  history:
    format: "csv"
    path: "data/sprint_history.csv"

tasks:
  - id: "discovery"
    name: "Discovery and architecture"
    planning_story_points: 5
```

The same project could instead reference a JSON file:

```
sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: "story_points"
  history:
    format: "json"
    path: "data/sprint_history.json"
```

1.8.5 External history file formats

For JSON, the simplest format is an array of objects using the same field names as the inline history rows:

```
[
  {
    "sprint_id": "SPR-2026-01",
    "sprint_length_weeks": 2,
    "completed_story_points": 21,
    "spillover_story_points": 5,
    "added_story_points": 3,
    "removed_story_points": 1,
    "holiday_factor": 1.0,
    "end_date": "2026-01-16",
    "team_size": 5,
    "notes": "Normal sprint"
  },
  {
    "sprint_id": "SPR-2026-02",
    "sprint_length_weeks": 2,
    "completed_story_points": 18,
    "spillover_story_points": 7,
    "added_story_points": 6,
    "removed_story_points": 2,
    "holiday_factor": 0.9,
    "end_date": "2026-01-30",
    "team_size": 5,
    "notes": "Production incident interrupted planned work"
  }
]
```

For CSV, the simplest format is a header row using the canonical field names from the inline schema:

```
sprint_id,sprint_length_weeks,completed_story_points,spillover_story_points,added_story_points,removed
SPR-2026-01,2,21,5,3,1,1.0,2026-01-16,5,Normal sprint
SPR-2026-02,2,18,7,6,2,0.9,2026-01-30,5,Production incident interrupted planned work
```

In both cases, omitted optional fields should still be allowed and should be normalized in the same way as inline rows after parsing.

1.8.6 Historical sprint data fields

The following table summarizes the fields that can be used inside each `sprint_planning.history` entry:

Name of field	Mandatory	Description
`sprint_id`	Yes	Stable identifier for the historical sprint row. This is the primary key used to store, validate, and report the record.
`sprint_length_weeks`	No	Sprint length for that historical observation. If omitted, it inherits from `sprint_planning.sprint_length_weeks`.
`completed_story_points`	Conditionally mandatory	Story points fully completed within the sprint. Exactly one of `completed_story_points` or `completed_tasks` must be present in each history row. If this field is used, the row's spillover, added, and removed values must use the `*_story_points` fields.
`completed_tasks`	Conditionally mandatory	Tasks or items fully completed within the sprint. Exactly one of `completed_story_points` or `completed_tasks` must be present in each history row. If this field is used, the row's spillover, added, and removed values must use the `*_tasks` fields.
`spillover_story_points`	No	Story points that were started or planned but not finished by sprint end. Use this only when the row is anchored by `completed_story_points`.
`spillover_tasks`	No	Tasks or items that spilled out of the sprint unfinished. Use this only when the row is anchored by `completed_tasks`.
`added_story_points`	No	Story points added after sprint start because of scope churn or urgent demand. Use this only when the row is anchored by `completed_story_points`.
`added_tasks`	No	Tasks or items added after sprint start because of scope churn or urgent demand. Use this only when the row is anchored by `completed_tasks`.
`removed_story_points`	No	Story points explicitly removed from the sprint after sprint start. Use this only when the row is anchored by `completed_story_points`.
`removed_tasks`	No	Tasks or items explicitly removed from the sprint after sprint start. Use this only when the row is anchored by `completed_tasks`.
`holiday_factor`	No	Optional capacity-scaling model input describing how much effective working time was available in that sprint relative to a normal sprint.

Name of field	Mandatory	Description
`end_date`	No	Optional metadata recording when the sprint ended. Useful for reporting, chronology hints, and later analysis, but not for identifying the row.
`team_size`	No	Optional metadata recording the team size for that sprint. Preserved for diagnostics and future analysis.
`notes`	No	Optional free-text metadata for contextual notes about the sprint.

When external history files are used instead of inline rows, JSON object keys and CSV column headers should use these same canonical field names.

Field omission note:

- `sprint_id` is mandatory for every historical sprint row and has no default.
- Exactly one of `completed_story_points` or `completed_tasks` must be present in every historical sprint row.
- Once one completed-unit field is chosen for a row, any `spillover_*`, `added_*`, and `removed_*` values in that row must use the same unit family.
- Any other historical sprint field may be omitted.
- If `sprint_length_weeks` is omitted in a history row, it should default to the parent `sprint_planning`
- If `completed_*`, `spillover_*`, `added_*`, or `removed_*` is not present in a history row, it should be treated as `0` for that row.
- If `removed_work_treatment` is omitted, it should default to `churn_only`.
- If `planning_confidence_level` is omitted, it should default to the documented project default, proposed here as `0.80`.
- If `future_sprint_overrides` is omitted, no explicit future sprint adjustments should be applied.
- If `holiday_factor` is omitted, it should default to `1.0`, meaning the sprint is treated as having normal available working time.
- If `end_date`, `team_size`, or `notes` is omitted, it should default to a null/ignored value and should not affect the forecast.

Example sparse historical row with omitted fields:

```
history:
  - sprint_id: "SPR-2026-01"
    completed_story_points: 20
```

This sparse row should be interpreted as if it had been written as:

```yaml
history:
  - sprint_id: "SPR-2026-01"
    sprint_length_weeks: 2          # inherited from sprint_planning.sprint_length_weeks
    completed_story_points: 20
    spillover_story_points: 0
    added_story_points: 0
    removed_story_points: 0
    holiday_factor: 1.0
    end_date: null
    team_size: null
    notes: null
```

Minimum usable-history note:

- Sparse rows are allowed for convenience, but the simulation should only run when the history contains at least two usable observations with positive delivery signal after defaulting and normalization.
- A practical MVP rule is to require at least two historical rows where normalized `completed_* > 0` or where attempted delivery signal (`completed_* + spillover_*`) is positive.

1.8.7 What `holiday_factor` means

`holiday_factor` is an optional per-history-row capacity scaling value that describes how much effective working time was available in that sprint relative to a normal sprint.

Recommended interpretation:

- `holiday_factor = 1.0`: normal sprint capacity, with no holiday-related reduction;
- `holiday_factor < 1.0`: reduced effective capacity because of public holidays, planned shutdowns, company events, or similar non-working time;
- `holiday_factor > 1.0`: unusually high effective capacity, which should be allowed but should normally be rare and used cautiously.

Examples:

- `holiday_factor = 0.9` means the sprint had about 90% of the normal working availability;
- `holiday_factor = 0.75` means the sprint had materially reduced working time and should not be treated as directly comparable to a full-capacity sprint.

1.8.8 Purpose of `holiday_factor`

The purpose of `holiday_factor` is to separate **calendar-driven capacity reduction** from **planning instability**.

Without this field, a sprint with lower completion due to holidays can look statistically similar to a sprint with lower completion due to poor planning, excessive spillover, or large amounts of urgent added work. That would pollute the historical learning signal. `holiday_factor` allows the model to recognize that some lower-output sprints were constrained by reduced availability rather than by

estimation error or delivery volatility.

This makes the historical data cleaner in three ways:

1. it reduces the risk of underestimating normal sprint capacity because holiday-affected sprints are mixed in without adjustment;
2. it reduces the risk of overstating spillover or churn as the explanation for low completion when the real cause was less available working time;
3. it gives the forecast a better basis for adjusting specific future sprints that are already known to have reduced working availability.

1.8.9 How `holiday_factor` should affect planning and prediction

`holiday_factor` should affect the model as a **capacity normalizer and forward-sprint adjustment**, not as churn.

For historical rows, the cleanest use is to normalize **delivery-side capacity signals** back to a nominal full-capacity basis before comparing or resampling them:

$$\text{normalized_completed}_i = \frac{\text{completed_units}_i}{\max(\text{holiday_factor}_i, \epsilon)}$$

$$\text{normalized_spillover}_i = \frac{\text{spillover_units}_i}{\max(\text{holiday_factor}_i, \epsilon)}$$

$$\text{normalized_added}_i = \text{added_units}_i, \quad \text{normalized_removed}_i = \text{removed_units}_i$$

This keeps a holiday-shortened sprint from being interpreted as evidence that the team normally completes less work. Added and removed scope should remain raw churn signals, because they describe priority change and replanning rather than available working time.

For future sprints, the same factor should be applied in the forward direction when a known future sprint has reduced availability:

$$\text{effective_future_capacity}_t = \text{sampled_nominal_capacity}_t \cdot \text{override_holiday_factor}_t$$

where override holiday factor t comes from a matching future sprint overrides entry when one is present, and otherwise defaults to 1.0.

This affects planning and prediction in three specific places:

1. **Sprint planning:** recommended planned load for a future sprint should be reduced when that sprint has a known `holiday_factor < 1.0`.
2. **Completion-date forecasting:** projected completion dates should move later when one or more future sprints are known to have reduced availability.

3. **Historical interpretation:** low-output historical sprints with reduced availability should be discounted as evidence of delivery instability.

In practical terms, `holiday_factor` should influence the forecast differently from the other historical fields:

- `spillover_*`, `added_*`, and `removed_*` describe sprint churn and planning instability;
- `holiday_factor` describes the amount of working time available to the team.

Those are not the same phenomenon and should not be merged statistically.

1.8.10 Support two planning units, but keep them separate

Story-point mode should use observed historical **completed story points**, **spillover story points**, **added story points**, and **removed story points** per sprint, and should consume backlog in story points. This is appropriate when the team already estimates backlog items in story points.

Task mode should use observed historical **completed tasks/items**, **spillover tasks/items**, **added tasks/items**, and **removed tasks/items** per sprint, and should consume backlog in units of task-count. This is appropriate only when the project tasks represent right-sized backlog items rather than large epics; otherwise the forecast will be misleading because one "task" may be far larger than another.

In this proposal, **right-sized** means backlog items are small and similar enough that completed item count is a stable planning signal. Scrum.org describes this in terms of a **Service Level Expectation (SLE)**: a statistical expectation for how long comparable work items take to finish. In practical `mcprojsim` terms, task-count mode is appropriate only when tasks are consistently small, comparable, and not acting as coarse-grained epics.

The current estimate model already supports multiple sizing styles, but that does **not** mean hours-based task estimates can safely be converted into story points, because story points are a relative team-specific measure rather than a time unit. Therefore:

- if `capacity_mode == "story_points"`, require each planned item to expose planning points explicitly, or require a separate `planning_story_points` field if the task's duration estimate is not already story-point based. In practice, every task must have a resolvable story-point value before story-point mode can run;
- if `capacity_mode == "tasks"`, count each eligible task as one item, but warn when item sizes are obviously heterogeneous.

1.8.11 Use a dependency-aware sprint pull simulator

The model is based on working on a subset of tasks from the project in each sprint. The right way to express that in this design is to build a **SprintPlanner** that mirrors the product's existing scheduling approach at a sprint level: it should maintain a ready queue of dependency-satisfied tasks and select from that queue sprint by sprint.

Here, the **ready queue** means the set of tasks whose declared dependencies are already complete. Priority is optional, but when a priority field is present it should be used to order pulls. A simple and explainable rule is to pull lower numeric priorities first and break ties by task ID.

When no priority field is present, the planner should still use a deterministic stable ordering, with task ID as the default tie-break and fallback ordering key. That keeps results reproducible across implementations and across repeated runs with the same random seed.

This section is also where the document first distinguishes two different mechanisms that can cause unfinished work to move into a later sprint:

- **Capacity-driven deferral**: the task is never started because it does not fit in remaining sprint capacity.
- **Execution spillover**: the task is started, consumes capacity, and still misses sprint end.

Recommended per-iteration algorithm:

1. Build the initial ready queue from tasks whose dependencies are already satisfied.
2. Sample the sprint outcome profile for Sprint `n`, including completed-capacity, added-work, removed-work, and churn behavior.
3. Pull ready tasks in priority order until the usable sprint capacity would be exceeded.
4. Inject sampled added work and remove sampled de-scoped work according to the configured interpretation.
5. Mark finished tasks complete at sprint end and carry forward unfinished work.
6. Unlock newly ready tasks.
7. Repeat until all tasks are done.
8. Record the number of sprints and the sprint-end date.

This produces a true distribution of **sprints-to-done**, rather than merely dividing a scalar backlog by scalar capacity.

1.8.12 Make empirical resampling the default uncertainty model

The default sprint-capacity generator should be **empirical bootstrap/resampling** from observed historical sprint outcomes, because that preserves actual volatility and avoids assuming a distribution shape the team may not have.

For example:

- in story-point mode, resample from historical quadruples of completed story points, spillover story points, added story points, and removed story points per sprint;
- in task mode, resample from historical quadruples of completed tasks, spillover tasks, added tasks, and removed tasks per sprint.

If the user changes `sprint_length_weeks`, the model should still treat the **sprint as the primary statistical entity**. A sprint has internal structure such as planning cadence, batching, review deadlines, and end-of-sprint completion effects, so breaking it into synthetic independent weeks is

not fully representative. The preferred approach is therefore to resample whole sprint observations whenever same-length historical cadence is available.

For mixed historical cadences or requested sprint lengths that do not appear in the history at all, the MVP can still fall back to **weekly outcome-rate normalization** as a compatibility mechanism. That fallback is useful, but it should be described explicitly as an approximation rather than as the ideal long-term method.

Recommended normalization:

$$\text{weekly_completed}_i = \frac{\text{completed_units}_i}{\text{sprint_length_weeks}_i}$$

$$\text{weekly_spillover}_i = \frac{\text{spillover_units}_i}{\text{sprint_length_weeks}_i}$$

$$\text{weekly_added}_i = \frac{\text{added_units}_i}{\text{sprint_length_weeks}_i}, \quad \text{weekly_removed}_i = \frac{\text{removed_units}_i}{\text{sprint_length_weeks}_i}$$

$$\text{simulated_sprint_outcome}(L \text{ weeks}) = \sum_{j=1}^{L}(\text{weekly_completed}_j + \text{weekly_spillover}_j + \text{weekly_added}_j +$$

This fallback is better than pretending that historical 2-week sprint capacity can be reused unchanged for a future 3-week sprint, but it should not be mistaken for a fully faithful representation of sprint behavior.

1.8.13 Treat spillover, scope-addition, and scope-removal as first-class historical signals

Historical sprint learning should not be limited to what was completed. Each historical row should be treated as a **joint outcome vector**:

$$X_i = (c_i^{\text{complete}}, s_i, a_i, r_i)$$

where:

- c_i^{complete} is the work fully finished inside sprint i;
- s_i is the work still carried out of sprint i at sprint end because it did not land;
- a_i is the work added after sprint start because higher-priority demand interrupted the original plan;
- r_i is the work explicitly de-scoped from sprint i after sprint start, either because priorities changed or because the plan proved unrealistic.

The most statistically sound default is to resample these vectors **jointly**, not as four independent series. Joint bootstrap preserves the observed correlation structure between healthy sprints, high-churn sprints, and unstable sprints. If the team historically sees that urgent scope additions and de-scoping tend to coincide with more spillover and lower completion, the simulator should preserve that relationship rather than averaging it away.

Two derived measures should then be computed from the same history:

$$\text{spillover_ratio}_i = \frac{s_i}{\max(c_i + s_i, \epsilon)}, \quad \text{scope_addition_ratio}_i = \frac{a_i}{\max(c_i + a_i, \epsilon)}, \quad \text{scope_removal_ratio}$$

These are the most useful normalized measures of planning instability:

- spillover ratio measures how much of the attempted sprint scope failed to land;
- scope addition ratio measures how much of the sprint demand arrived after planning was supposedly complete;
- scope removal ratio measures how much planned sprint scope had to be taken back out after sprint start.

Those historical signals should be used in two different ways:

1. **Completion-date forecasting:** added work should be treated as stochastic scope growth, removed work should be treated as stochastic scope shrinkage, and spillover should calibrate how much of the sprint plan actually lands versus carries forward.
2. **Planned-load guidance:** future sprint commitments should be lower than raw historical completion whenever the historical data shows frequent scope additions, heavy spillover, or repeated de-scoping of planned work.

Historical removed work needs a slightly more careful interpretation than historical added work. Added work nearly always represents genuine extra demand that consumed capacity or expanded backlog. Removed work can mean one of two things:

1. the team legitimately discovered that some planned work should no longer be done at all; or
2. the team used removal as a planning escape hatch to avoid reporting spillover.

The model should therefore use removed work in **two** ways:

- as a measure of **planning churn**, because frequent removals mean the sprint plan was unstable;
- as an optional source of **backlog shrinkage**, but only when the user intends historical removals to represent genuine descoping rather than reclassification.

The cleanest design is to add an optional interpretation flag at the planning-spec level, for example `removed_work_treatment: "churn_only" | "reduce_backlog"`, defaulting to `"churn_only"`. That keeps the forecast conservative unless the user explicitly states that removed work should be modeled as real reduction in remaining project scope.

The simplest commitment rule that makes good use of the available data is:

$$\text{recommended_planned_commitment}(q) = \max\left(0, Q_{50}(\text{completed_units}) \cdot (1 - Q_q(\text{spillover_ratio})) \cdot \right.$$

for a chosen planning confidence level q such as 0.80. This rule is deliberately conservative: it starts with typical delivered capacity, discounts it by a high-percentile spillover rate, discounts again for the historical tendency to remove planned work after sprint start, and then reserves explicit room for likely mid-sprint scope additions.

This commitment rule does not contradict the earlier recommendation against average-velocity forecasting. It is a planning heuristic derived from historical percentiles, not the primary forecasting engine. Completion-date forecasts should still come from full empirical resampling rather than from a single collapsed point estimate.

For completion-date forecasting, the simulator should instead use the richer sprint recursion:

$$B_{t+1} = B_t - D_t + A_t - R_t^{\text{effective}}$$

where D_t is delivered units, produced by the dependency-aware planner after applying sampled capacity and sampled spillover behavior, and $R_t^{\text{effective}}$ is either 0 in churn only mode or the sampled removed work in reduce backlog mode. This is the better place to use the new data for forecast dates, because it models backlog growth, backlog shrinkage, and sprint instability directly instead of collapsing them into one average burn rate.

1.8.14 Add an explicit volatility layer, but keep it optional

Historical resampling already captures ordinary variation, but the user also asked for "some measurement of the uncertainty in the sprint capacity and potential volatility." The best design is to distinguish:

- **uncertainty** = the spread already visible in historical sprint outcomes;
- **volatility** = exogenous future shocks that may reduce capacity beyond the normal observed range.

The current engine already uses multiplicative uncertainty factors on task duration. A sprint mode can use the same design pattern, but it should apply it specifically to the **deliverable-capacity component** of the sampled sprint outcome rather than to every historical variable indiscriminately:

$$extef\,fective_deliverable_capacity = \text{sampled_completed_capacity} \cdot \text{sampled_volatility_multiplier}$$

where the multiplier defaults to 1.0 when no extra volatility model is enabled.

In the MVP design, the volatility overlay should affect the forecast as follows:

- it scales deliverable capacity for the sprint;
- it does **not** directly rescale sampled `added_units` or `removed_units`, because those are scope-churn signals rather than capacity signals;
- it does **not** directly overwrite sampled spillover history, because spillover should primarily emerge from reduced effective delivery capacity and the task-level spillover model.

That boundary keeps the statistical model coherent: calendar and disruption effects reduce what the team can deliver, while churn variables continue to represent changing scope and replanning behavior.

Operationally, the two uncertainty layers should be applied in sequence. First, the volatility overlay adjusts effective sprint capacity for the sprint. Then, once tasks are actually pulled into that sprint, the execution-spillover model evaluates whether any pulled task overruns and carries work forward. The two mechanisms are intentionally independent.

Recommended volatility options:

1. **Empirical only (default)**: no extra layer; use history as-is.
2. **Empirical + disruption overlay**: with probability `p_disruption`, multiply sprint capacity by a sampled factor below 1.
3. **Scenario-driven overrides**: allow planned PTO/known holidays/future events to reduce specific future sprints.

This preserves a simple mental model for users while still allowing "normal" capacity variation and "shock" variation to be reported separately. ### Model task-level execution uncertainty and spillover

The capacity-volatility layer in section 6 captures sprint-team-wide capacity variation. A second, independent source of uncertainty is **task-level execution overrun**: a task is selected into a sprint, consumes capacity during the sprint, but is not completed by sprint end. It therefore contributes zero to delivered throughput while still consuming some or all of the sprint's capacity budget for that item.

This is distinct from the case in which a task simply does not fit into remaining capacity and is deferred to a later sprint without being started. Execution spillover means the task *starts*, runs over its planned size, and carries unfinished work into the next sprint.

Why spillover probability should be size-dependent

Empirically, smaller tasks are more predictable: their actual effort stays close to their estimate, so they either finish cleanly within the sprint or are held back. Larger tasks have higher uncertainty in absolute terms, so even when planned-vs-actual ratios are similar, a larger task has more absolute room to overrun. The design should therefore treat spillover probability as a monotonically increasing function of a task's planned effort, so an 8 SP task has a materially higher probability of spilling over than a 3 SP task.

A practical model is a logistic function of planned size:

$$P(\text{spillover} \mid s) = \frac{1}{1 + \exp(-(a \log(s/\text{ref_size}) + b))}$$

where ref size is a reference task size (e.g. 5 SP), a controls the steepness of the increase with size, and b shifts the overall base rate. A simpler piecewise-linear approximation is also acceptable if user-facing calibration is preferred over a continuous curve:

Planned size (SP)	Default spillover probability
2	0.05
3–5	0.12
6–8	0.25
> 8	0.40

Both forms should be overridable via config; the table-based form is the recommended default for explainability.

This execution-spillover model depends on a task-size signal expressed in planning story points. In `story_points` mode that size already exists as part of sprint planning. In `tasks` mode the same model should only be enabled when each eligible task also exposes an explicit planning story-point size for spillover calibration; otherwise the feature should be disabled or rejected at validation time. Task-count mode therefore remains valid for sprint-capacity forecasting even when execution-spillover modeling is unavailable.

How overrun effort is modeled

When a spillover event is triggered for a task, the sprint consumes only a sampled fraction of the task's planned effort during the current sprint. The remaining effort carries forward as a reduced-size "remainder task" that re-enters the ready queue for the next sprint (retaining all original dependencies, now satisfied):

$$F \sim \text{Beta}(\alpha_{\text{consumed}}, \beta_{\text{consumed}}), \quad \text{remaining_effort} = \text{planned_effort} \cdot (1 - F)$$

The default Beta distribution should have $\alpha_{\text{consumed}} = 3.25$, $\beta_{\text{consumed}} = 1.75$, giving a mean consumed fraction of roughly 0.65 (most of the work is done, but the task still does not land). These values should be treated as pragmatic starter defaults rather than universalconstants. Teams should tune them against their own historical carryover patterns if such data is available. This parameterisation also captures the occasional case where a task was barely started before it was recognized as too large to finish, by assigning some probability to low consumed fractions.

Meaning of `consumed_fraction_alpha` and `consumed_fraction_beta`

Both parameters belong to the Beta distribution used to sample `fraction_consumed` when a spillover

event occurs. They are shape parameters, so they control both the central tendency and dispersion of how much of a spilled task is completed before sprint end.

Parameter	What it controls	Valid range	Default	Effect when increased (holding the other fixed)
`consumed_fraction_alpha`	Pulls the sampled consumed fraction toward `1.0` (more work finished before spillover)	`> 0`	`3.25`	Increases expected consumed fraction and generally reduces remainder size
`consumed_fraction_beta`	Pulls the sampled consumed fraction toward `0.0` (less work finished before spillover)	`> 0`	`1.75`	Decreases expected consumed fraction and generally increases remainder size

Useful identities for interpretation:

- Mean consumed fraction: `E[fraction_consumed] = alpha / (alpha + beta)`
- Variance: `Var[fraction_consumed] = alpha * beta / ((alpha + beta)^2 * (alpha + beta + 1))`

With the defaults, `E[fraction_consumed] = 3.25 / (3.25 + 1.75) = 0.65`, so the model expects about 65% of a spilled task to be consumed in the sprint where spillover happens, with the remaining ~35% carried into a later sprint.

Capacity accounting

During the sprint, the consumed fraction of the spilled task is charged against sprint capacity just like a completed task. The net effect is:

- sprint capacity is partially or fully exhausted by the spilled task;
- no throughput credit is awarded for that sprint;
- the remainder task enters the next sprint's ready pool;
- diagnostics should record carryover load per iteration so the aggregate spillover distribution can be reported.

Config shape

```yaml
sprint_planning:
  spillover:
    enabled: true
    model: "table"           # "table" | "logistic"
    size_reference_points: 5
    # table mode: one entry per size bracket
    size_brackets:
      - max_points: 2
        probability: 0.05
      - max_points: 5
        probability: 0.12
      - max_points: 8
        probability: 0.25
      - max_points: null    # unbounded
        probability: 0.40
    consumed_fraction_alpha: 3.25
    consumed_fraction_beta: 1.75
```

1.8.15 Project-file parameter clarifications

The following parameters appear in the project-file examples and are required to be interpreted consistently by implementations.

Parameter	Location	Meaning	Allowed values / constraints	Default when omitted
`uncertainty_mode`	`sprint_planning`	Selects how sprint outcomes are sampled from history.	`"empirical"` for MVP (bootstrap/resampling of observed sprint outcomes).	`"empirical"`
`planning_confidence_level`	`sprint_planning`	Confidence level used for planned-load guidance (not the core completion simulation iteration count).	Real number in `(0, 1)`, typically `0.50` to `0.95`.	`0.80`

Parameter	Location	Meaning	Allowed values / constraints	Default when omitted
`removed_work_treatment`	`sprint_planning`	Controls whether removed scope only informs churn diagnostics, or also reduces remaining backlog in the forecast recursion.	`"churn_only"` or `"reduce_backlog"`.	`"churn_only"`
`future_sprint_overrides`	`sprint_planning`	Optional list of known future sprint adjustments applied during forward simulation.	List of override objects.	No overrides
`future_sprint_overrides[].sprint_number`	override row	Target future sprint index (1-based) for the override.	Positive integer. Mutually exclusive with `start_date`.	`null`
`future_sprint_overrides[].start_date`	override row	Date-based target for an override when sprint number is not used.	ISO date string (`YYYY-MM-DD`). Mutually exclusive with `sprint_number`.	`null`
`future_sprint_overrides[].holiday_factor`	override row	Calendar-availability multiplier for that future sprint.	Real number `> 0`; typically `<= 1.0` for reduced availability.	`1.0`
`future_sprint_overrides[].capacity_multiplier`	override row	Explicit capacity multiplier for that future sprint, applied to sampled nominal sprint capacity.	Real number `> 0`. If both this and `holiday_factor` are present, multiply them.	`1.0`

Parameter	Location	Meaning	Allowed values / constraints	Default when omitted
`volatility_overlay.enabled`	`sprint_planning.volatility_overlay`	Enables stochastic sprint-level disruption on top of empirical sampling.	Boolean.	`false`
`volatility_overlay.disruption_probability`	`sprint_planning.volatility_overlay`	Probability that a sprint receives a disruption multiplier draw.	Real number in `[0, 1]`.	`0.0`
`volatility_overlay.disruption_multiplier_low`	`sprint_planning.volatility_overlay`	Lower bound for disruption multiplier sampling.	Real number `> 0`; with MVP disruption semantics: `low <= expected <= high <= 1.0`.	`1.0`
`volatility_overlay.disruption_multiplier_expected`	`sprint_planning.volatility_overlay`	Most likely disruption multiplier value (mode).	Real number `> 0`; ordered between `low` and `high`.	`1.0`
`volatility_overlay.disruption_multiplier_high`	`sprint_planning.volatility_overlay`	Upper bound for disruption multiplier sampling.	Real number `> 0`; with MVP disruption semantics typically `<= 1.0`.	`1.0`
`spillover.enabled`	`sprint_planning.spillover`	Enables task-level execution spillover modeling.	Boolean.	`false`
`spillover.model`	`sprint_planning.spillover`	Spillover-probability function family.	`"table"` or `"logistic"`.	`"table"`
`spillover.size_reference_points`	`sprint_planning.spillover`	Reference size used by logistic model (`ref_size` in the formula).	Real number `> 0`.	`5`

Parameter	Location	Meaning	Allowed values / constraints	Default when omitted
`spillover. size_brackets`	`sprint _planning. spillover`	Ordered piecewise probability definition used by table model.	Ascending `max_points` thresholds; final catch-all may use `null`. Each `probability` in `[0,1]`.	Built-in default brackets (2, 5, 8, null) with probabilities (0.05, 0.12, 0.25, 0.40)
`spillover. size_brackets[]. max_points`	bracket row	Upper size bound included in the bracket.	Positive number or `null` for unbounded final bracket.	n/a
`spillover. size_brackets[]. probability`	bracket row	Spillover probability for tasks in that bracket.	Real number in `[0, 1]`.	n/a
`spillover. consumed _fraction _alpha`	`sprint _planning. spillover`	Beta-shape parameter pulling consumed fraction toward `1.0`.	Real number `> 0`.	`3.25`
`spillover. consumed _fraction _beta`	`sprint _planning. spillover`	Beta-shape parameter pulling consumed fraction toward `0.0`.	Real number `> 0`.	`1.75`

Interpretation notes:

- `planning_confidence _level` affects commitment guidance (how much to plan into a future sprint), while completion-date forecasting still comes from full Monte Carlo recursion.
- `removed_work _treatment = "churn_only"` keeps removed work out of backlog shrinkage; `"reduce_b` treats removed work as real backlog reduction in the forecast recursion.
- For `future_sprint _overrides`, exactly one targeting key should be provided per row: `sprint_numb` or `start_date`.
- When `volatility_overlay. enabled = false`, the effective disruption multiplier is `1.0`.
- In `spillover.model = "table"`, `size_reference _points` is ignored; in `spillover.model = "logisti` `size_brackets` is ignored.

1.9 Measuring Uncertainty and Volatility

The project already reports standard deviation and coefficient of variation (CV) for simulated duration outputs. The sprint feature should report analogous metrics for historical sprint outcome diagnostics and for the simulated sprints-to-done forecast:

- mean capacity,
- median capacity,
- standard deviation,
- coefficient of variation (`std_dev / mean`),
- P10 / P50 / P90 capacity,
- mean spillover units,
- spillover ratio percentiles,
- mean added units,
- scope-addition ratio percentiles,
- mean removed units,
- scope-removal ratio percentiles,
- correlation between completed, spillover, added, and removed units,
- disruption frequency (if volatility overlay is enabled).

These should be split into two views:

1. the **input historical sprint-outcome series**, and
2. the **simulated sprints-to-done output**.

Good user-facing diagnostics would include:

- "historical story-point capacity: mean 21.4, CV 0.18";
- "historical spillover ratio: P50 0.12, P80 0.24";
- "historical added scope: mean 3.1 SP, CV 0.44";
- "historical removed scope: mean 1.4 SP, P80 ratio 0.09";
- "recommended planned load at P80 confidence: 15 SP";
- "P80 completion: 6 sprints";
- "10% modeled probability of disrupted sprint, median disruption multiplier 0.72".

1.10 How to Handle Adjustable Sprint Lengths

Sprints are fixed-length events in Scrum, but the chosen fixed length can differ between teams or scenarios. The safest implementation is:

1. allow a configured `sprint_length_weeks` as a whole number;
2. prefer resampling whole historical sprint observations at the same cadence as the configured sprint length;
3. if the requested cadence is not available in history, allow an MVP fallback that normalizes historical sprint outcomes to weekly rates and re-aggregates them to the chosen sprint length,

with an explicit warning that this is an approximation.

If enough history exists for multiple sprint lengths, the better approach is to segment history by sprint length and prefer same-length history before falling back to weekly normalization. That preserves cadence-specific effects while still supporting scenario analysis.

The preferred advanced extension after the MVP is to use **block bootstrap over whole sprint observations** rather than synthetic week-by-week recombination. That keeps the sprint intact as the forecasting entity and better preserves internal sprint structure.

If the requested sprint length does not appear in the historical data at all, the system should still allow forecasting by falling back to weekly normalization and re-aggregation, but it should warn that the chosen cadence is being extrapolated from different historical sprint lengths.

For delivery-date projection, map sprint counts to dates using sprint boundaries rather than working-day accumulation. However, known calendar effects can still be folded in by scaling future sprint capacity by the ratio of available workdays in that sprint versus nominal workdays.

1.11 How to Treat Partial Work and Carryover

There are two distinct ways a task can fail to land in a sprint; they require different modeling treatments.

Type 1: Capacity-driven deferral. The task was not pulled into the sprint because there was insufficient remaining capacity to start it. No effort is consumed and no capacity is charged; the task simply waits in the ready queue. This is the case addressed by non-preemptive whole-item pull: if the next ready task does not fit in remaining sprint capacity, it is deferred unchanged.

Type 2: Execution-driven spillover. The task was pulled into the sprint, effort was expended against it, but the work was not completed by sprint end. This is a separate phenomenon: capacity is partially or fully consumed, yet no throughput credit is awarded. The task carries its remaining effort into the next sprint as a reduced-size item.

Atlassian's velocity guidance says only **fully completed** stories count toward sprint velocity. Both types of carryover are consistent with that rule: deferred tasks contribute zero to velocity with zero capacity cost; spilled tasks contribute zero to velocity but do carry a capacity cost.

The default for this feature should be **non-preemptive whole-item pull with execution-driven spillover modeled separately** (see section 7):

- a task that does not fit in remaining capacity is left for a future sprint (Type 1: no capacity consumed);
- a task that is pulled but triggers a spillover event consumes a sampled fraction of sprint capacity and carries its remaining effort forward as a new, smaller task (Type 2);
- completed tasks are credited to throughput normally.

The two mechanisms combine naturally in the per-iteration algorithm:

1. For each ready task in pull order, if capacity remains for the full task, pull it and check for spillover using the size-dependent probability model.
2. If a spillover event fires, charge the consumed fraction, mark the remainder for the next sprint, and continue pulling other tasks with any remaining capacity.
3. If capacity would be exhausted before starting the full task, defer it without charging capacity.
4. At sprint end, compute delivered throughput from fully completed tasks only.

Diagnostics should record, per iteration, both the number of deferred tasks and the total remaining effort from spillover events, so the aggregate carryover distribution can be distinguished from capacity-driven deferral in reports.

Historical spillover, scope-addition, and scope-removal data should also shape how much work is intentionally loaded into the sprint in the first place. A planner that ignores these measures will systematically overcommit whenever the team has a history of churn or incomplete landings. The output should therefore include both:

- a **completion forecast** for the full backlog, and
- a **recommended planned-load range** for the next sprint, derived from the historical outcome vectors and the chosen planning confidence level.

1.12 Data Model Recommendation

1.12.1 New project-file structures

Recommended new models:

- `SprintPlanningSpec`
- `SprintHistoryEntry`
- `FutureSprintOverrideSpec`
- `SprintVolatilitySpec`
- `SprintSpilloverSpec`
- `SprintCarryoverRecord`
- `SprintPlanningResults`

Suggested fields:

Model	Fields
`SprintPlanningSpec`	`enabled`, `sprint_length_weeks`, `capacity_mode`, `history`, `uncertainty_mode`, `volatility_overlay`, `spillover`, `planning_confidence_level`, `removed_work_treatment`, `future_sprint_overrides`

Model	Fields
`SprintHistoryEntry`	`sprint_id`, `end_date?`, `sprint_length_weeks`, `completed_tasks?`, `completed_story_points?`, `spillover_tasks?`, `spillover_story_points?`, `added_tasks?`, `added_story_points?`, `removed_tasks?`, `removed_story_points?`, `team_size?`, `holiday_factor?`, `notes?`
`SprintHistorySourceSpec`	`format` (`json` or `csv`), `path`
`FutureSprintOverrideSpec`	`sprint_number?`, `start_date?`, `holiday_factor?`, `capacity_multiplier?`, `notes?`
`SprintVolatilitySpec`	`enabled`, `disruption_probability`, `disruption_multiplier_low`, `disruption_multiplier_expected`, `disruption_multiplier_high`
`SprintSpilloverSpec`	`enabled`, `model` (`table` or `logistic`), `size_reference_points`, `size_brackets`, `consumed_fraction_alpha`, `consumed_fraction_beta`
`SprintCarryoverRecord`	`sprint_number`, `task_id`, `planned_points`, `consumed_fraction`, `remaining_points`
`SprintPlanningResults`	`sprint_counts`, `sprint_percentiles`, `date_percentiles`, `capacity_statistics`, `spillover_statistics`, `scope_addition_statistics`, `scope_removal_statistics`, `joint_outcome_statistics`, `planned_commitment_guidance`, `burnup_percentiles`, `carryover_statistics`

The new optional historical fields should be interpreted as follows:

- `sprint_id`: mandatory stable identifier for the historical sprint row; used as the primary key for storage, validation, and reporting.
- `completed_story_points` / `completed_tasks`: exactly one of these two fields must be present in each historical row, and that choice determines the unit family for the rest of the row.
- `removed_story_points` / `removed_tasks`: work explicitly taken out of the sprint after sprint start;
- `removed_work_treatment`: whether removed work only informs churn/commitment guidance or also reduces the remaining forecast backlog.
- `holiday_factor`: the fraction of normal working availability in that sprint, used to distinguish reduced calendar availability from true planning or delivery instability.

- `end_date`: optional metadata for reporting, chronology hints, and potential later segmentation; not required for the MVP forecast math and not the primary key for the row.
- `team_size`: optional metadata for diagnostics and future analysis; not used as a direct scaling input in the MVP forecast.
- `future_sprint_overrides`: optional forward-looking calendar/capacity adjustments for specific known future sprints.
- `history` may be either an inline list of `SprintHistoryEntry` rows or a `SprintHistorySourceSpec` pointing to an external JSON or CSV history file.

All historical sprint-row fields other than `sprint_id` should be optional per history row. Neutral defaults should be applied as follows:

- `sprint_id` -> required, no default
- `sprint_length_weeks` -> inherit from `SprintPlanningSpec.sprint_length_weeks`
- `completed_*` -> `0`
- `spillover_*` -> `0`
- `added_*` -> `0`
- `removed_*` -> `0`
- `holiday_factor` -> `1.0`
- `end_date` -> `null` / ignored
- `team_size` -> `null` / ignored
- `notes` -> `null` / ignored

An implementation-oriented schema defaulting rule should be stated explicitly:

- every history row must include a non-empty `sprint_id`, and `sprint_id` values should be unique within the provided history list;
- every history row must include exactly one of `completed_story_points` or `completed_tasks`;
- once a row selects a completed-unit field, all `spillover_*`, `added_*`, and `removed_*` values in that row must use the same unit family;
- every omitted history-row field should be normalized to its neutral default before statistical processing;
- `sprint_length_weeks` inherits from the parent sprint-planning spec when omitted in a history row;
- `completed_*`, `spillover_*`, `added_*`, and `removed_*` default to `0` per history row;
- `holiday_factor` defaults to `1.0` per history row;
- `removed_work_treatment` defaults to `churn_only` at the planning-spec level;
- `planning_confidence_level` should default to a documented value such as `0.80` if omitted.

1.12.2 Task-level additions

If story-point sprint mode needs to coexist with time-based duration estimates, add a separate optional field such as `planning_story_points` on `Task`. That avoids conflating delivery forecasting units with time-estimation units.

Optional additions:

- `priority` for pull order within the ready queue,
- `epic` / `milestone` grouping for richer burn-up views,
- `sprint_sized: true/false` to gate task-throughput mode warnings,
- `spillover_probability_override: float` to allow per-task overrides of the size-bracket default when a specific task is known to carry higher execution risk.

1.13 Recommended Internal Component Design

Component	Responsibility	Suggested module
`SprintHistoryFileParser`	Load external JSON and CSV sprint-history files and normalize them into canonical history rows before validation/defaulting	`parsers/sprint_history_parser.py`
`SprintCapacitySampler`	Jointly sample per-sprint completed, spillover, added-work, and removed-work outcomes from history and volatility config	`planning/sprint_capacity.py`
`SprintPlanner`	Build ready queue, pull tasks into sprint buckets, inject unplanned additions, remove de-scoped work, and advance dependencies	`planning/sprint_planner.py`
`SprintSimulationEngine`	Run N iterations, update backlog with added and optionally removed work, and produce sprint-count arrays	`planning/sprint_engine.py`
`SprintPlanningResults`	Percentiles, dates, capacity stats, churn metrics, and planned-load guidance	`models/sprint_simulation.py`

This keeps sprint planning modular and avoids destabilizing the current duration-based simulation engine.

1.14 Integration with Existing `mcprojsim` Concepts

1.14.1 Reuse what already fits

- Reuse the existing Monte Carlo iteration pattern.

- Reuse the existing statistics and percentile reporting style.
- Reuse the existing project dependency graph and task order.
- Reuse calendar metadata to adjust future sprint capacity when desired.

1.14.2 Do not force-fit what does not

- Do not convert hours directly into story points.
- Do not reuse critical-path elapsed duration as sprint count; sprint planning is a backlog-capacity forecast, not a path-length forecast.
- Do not assume task-count throughput is valid unless items are right-sized.

1.15 MVP vs. Follow-On Roadmap

This proposal is intended to be decision-ready for the MVP scope. The later phases below are included as future-work guidance, not as part of the MVP commitment.

1.15.1 MVP

1. Add `sprint_planning` project schema.
2. Support `capacity_mode = story_points | tasks`.
3. Add historical `spillover_*`, `added_*`, and `removed_*` fields to sprint history.
4. Implement empirical joint resampling of completed, spillover, added-work, and removed-work history.
5. Implement dependency-aware sprint pulling with whole-item completion.
6. Report `P50/P80/P90 sprints`, delivery dates, recommended planned load, and historical capacity/churn stats.

1.15.2 Phase 2

1. Add optional volatility/disruption overlay.
2. Add execution-driven spillover modeling with size-dependent probability and sampled consumed-fraction carryover (see section 7). Report per-sprint carryover distribution alongside the main sprints-to-done percentiles.
3. Add calendar-adjusted future sprint capacity.
4. Add burn-up percentile charts by sprint.
5. Add warnings for heterogeneous task sizes in task-throughput mode.
6. Add support for loading sprint history from external JSON and CSV files referenced from the main project file.

The phased roadmap above defines delivery sequencing only. The formal requirements below describe the full target-state feature set; Phase 2 and Phase 3 items therefore remain part of the overall specification, but they are not required for the first MVP increment unless explicitly marked into scope for that increment.

1.15.3 Phase 3

1. Add parametric capacity models as advanced options.
2. Add scenario analysis for team-size changes.
3. Add support for multiple teams / multiple sprint lanes.

1.16 Implementation Notes

To make the proposal implementable without further design gaps, the following details should be treated as part of the MVP-level document guidance:

1. **Schema and validation behavior**

- `SprintHistoryEntry` should accept sparse historical rows where churn fields are absent.
- `SprintPlanningSpec.history` should accept either inline rows or an external history source descriptor, but not both at the same time.
- `SprintHistoryEntry` should require a non-empty `sprint_id` for every historical row and treat that field as the row's primary identifier.
- `SprintHistoryEntry` should require exactly one of `completed_story_points` or `completed_tasks` in every historical row.
- Validation should normalize every omitted history-row field to its neutral default before downstream statistical processing.
- Validation should reject duplicate `sprint_id` values within the same history series.
- Validation should reject history rows that mix task-based and story-point-based fields in the same row.
- Validation should reject histories that do not contain at least two usable observations with positive delivery signal after defaulting and normalization.
- Validation should reject mixed unit families within the same history series.
- External JSON history files should contain an array of objects using the canonical history-row field names.
- External CSV history files should contain a header row using the canonical history-row field names as column names.
- External history files should be parsed into the same internal row representation as inline history before downstream validation and normalization.
- Validation should reject non-positive `holiday_factor` values in historical rows and in future sprint overrides.
- Validation should reject task-level spillover configurations that lack a resolvable planning story-point size for every task eligible for spillover evaluation.

`end_date` and `team_size` should be treated as metadata-only fields in the MVP. They may be stored and reported, but they should not silently alter capacity calculations unless a later design explicitly introduces that behavior. `end_date` should no longer act as the primary identifier for a historical sprint row.

2. **Sampler behavior**

`SprintCapacitySampler` should normalize historical rows into a canonical internal tuple:

$$(c, s, a, r)$$

- The sampler should apply per-row default filling before any resampling or fallback normalization.
- The sampler should only holiday-normalize delivery-side quantities (`completed`, and optionally `spillover` if that interpretation is retained), while leaving `added` and `removed` as raw churn signals.
- The sampler should expose both the sampled delivered-capacity component and the sampled churn components so downstream planning logic does not have to reverse-engineer them from a single scalar.
- The sampler should also apply any matching `future_sprint_overrides` when simulating known future sprints.
- The sampler should treat whole historical sprint rows as the primary resampling entity whenever same-length cadence data exists, and only use week-level normalization as an explicit fallback for mixed-cadence extrapolation.

3. **Planner and engine behavior**

- `SprintPlanner` should treat added work and removed work as sprint-level events distinct from task execution spillover.
- `SprintPlanner` should use a deterministic pull order: lower numeric priority first when `priority` is present, otherwise task ID order.
- `SprintSimulationEngine` should make the backlog update rule explicit and auditable per iteration so it is clear whether a date forecast changed because of delivery, spillover, added scope, or removed scope.
- `SprintSimulationEngine` should represent sampled aggregate added work and removed work as explicit auditable backlog adjustments or equivalent ledger entries rather than silently burying them inside capacity accounting.
- Volatility should reduce deliverable capacity, while churn and task-level spillover should remain behaviorally separate mechanisms.

4. **CLI and export behavior**

- CLI summaries should surface the selected `removed_work_treatment` whenever sprint planning mode is active.
- JSON and CSV exports should include raw historical churn summaries, derived ratios, and recommended planned-load guidance in a dedicated sprint-planning section.
- Reports should clearly distinguish:
  - work completed,
  - work added,
  - work removed,
  - work spilled over,

  - and work still remaining.

These implementation notes do not change the design; they close the remaining specification gaps between the proposal and an actual implementation plan.

1.17 Why This Is the Best-Fit Approach for `mcprojsim`

This design fits the repository because it preserves the current Monte Carlo style, keeps the existing duration/effort outputs intact, and adds a parallel forecast for sprint cadence rather than trying to reinterpret the current duration arrays. It is also well grounded in agile practice: Sprints are fixed cadence, velocity/throughput should be based on completed work, and forecasts should reflect observed variation rather than a single mean.

Most importantly, it answers the actual user problem:

- adjustable sprint length in whole weeks,
- either tasks-per-sprint or story-points-per-sprint capacity,
- historical learning from completed work, spillover, and scope added during the sprint,
- historical learning from work removed from the sprint after planning,
- a statistically grounded recommendation for how much planned work should be loaded into future sprints,
- explicit uncertainty and volatility handling,
- and a percentile distribution for how many sprints it takes to finish the project.

1.18 Confidence Assessment

High confidence

- `mcprojsim` is already structurally compatible with a sprint Monte Carlo extension because it already samples work stochastically, stores per-iteration outputs, and reports percentile/statistical summaries.
- An empirical Monte Carlo sprint-capacity model is better founded than an average-velocity model, based on agile forecasting references.
- A dependency-aware sprint pull planner is the right way to model "subset of tasks per sprint" in this design.

Medium confidence

- Weekly normalization is a workable MVP fallback for adjustable sprint lengths, but it can underrepresent ceremony, batching, and end-of-sprint effects because a sprint is not well represented as a bag of independent weeks.
- Treating the sprint as the primary entity and using block bootstrap over whole sprint observations is the preferred advanced extension when the product moves beyond the MVP baseline.
- Task-count throughput mode is valuable, but only if tasks are roughly right-sized; some

projects in `mcprojsim` may currently define tasks at too coarse a granularity for that to be reliable.
- Historical added-work data tells us that scope churn exists, but if the project does not distinguish whether added work was also finished inside the same sprint, commitment guidance will remain conservative rather than exact.
- Historical removed-work data is useful, but its forecast meaning depends on whether removal means genuine descoping or only replanning; the default should therefore remain conservative.

Assumptions / inferred design choices

- I assume sprint planning is intended as an **additional** forecast view, not a replacement for the current duration/effort simulation.
- I assume "subset of tasks" means selecting whole tasks/items into sprints, not simulating arbitrary task fractions.
- I assume the product can introduce one or two new planning-specific fields on tasks if story-point sprint mode must coexist with hour-based duration estimates.

1.19 Footnotes

Chapter 2

Statistical Methods and Approach

This section consolidates the statistical model used throughout the proposal so the formal requirements can be read as implementation constraints rather than as the first place where the method is introduced. The guiding design choice is that sprint planning should remain an **empirical Monte Carlo forecast**. The model should preserve observed variation and observed coupling between delivery, spillover, and scope churn, rather than compressing history into a single average velocity.

2.1 Canonical Historical Observation

After schema validation and default filling, each historical sprint row should be transformed into a canonical observation:

$$X_i = (c_i, s_i, a_i, r_i, h_i, L_i)$$

where:

- c_i = completed units in sprint i
- s_i = historical spillover units in sprint i
- a_i = added units in sprint i
- r_i = removed units in sprint i
- h_i = holiday factor in sprint i
- L_i = sprint length in weeks for sprint i

The unit family is either story points or tasks, determined by the row's completed-unit field and the active `capacity_mode`. All missing churn fields are first normalized to `0`, and missing `holiday_factor` is normalized to `1.0`.

2.2 Delivery-Side Normalization

The model treats holiday factor as a delivery-capacity normalizer rather than as a churn signal. Historical sprints affected by reduced working availability should therefore be mapped back to a nominal full-capacity basis before they are compared or resampled.

Let ϵ be a small strictly positive constant used only to avoid divide-by-zero in formulas. Then the normalized historical quantities are:

$$\hat{c}_i = \frac{c_i}{\max(h_i, \epsilon)}$$

$$\hat{s}_i = \frac{s_i}{\max(h_i, \epsilon)}$$

$$\hat{a}_i = a_i, \quad \hat{r}_i = r_i$$

Only delivery-side quantities are holiday-normalized. Added work and removed work remain raw churn signals because they represent planning change rather than available working time.

2.3 Sprint-Entity Resampling and Mixed Sprint Lengths

Historical rows may use different sprint lengths. The preferred statistical stance is that the **sprint remains the primary observation unit**. A sprint has inherent internal structure, so it is not generally a good idea to break it up into independent synthetic weeks and assume the result is still representative.

When the configured sprint length matches a historical cadence that already exists in the data, the model should therefore resample **whole normalized sprint rows** as complete entities.

For mixed historical cadences, or when the configured sprint length does not appear in the available data, the MVP may still fall back to a weekly normalization approximation:

$$w_i^c = \frac{\hat{c}_i}{L_i}, \quad w_i^s = \frac{\hat{s}_i}{L_i}, \quad w_i^a = \frac{\hat{a}_i}{L_i}, \quad w_i^r = \frac{\hat{r}_i}{L_i}$$

For a simulated sprint length of `L` weeks, that fallback builds a simulated sprint outcome by summing `L` sampled weekly observations. This is acceptable as an MVP baseline for cross-cadence extrapolation, but it should be documented as an approximation rather than as the preferred long-term model.

The preferred advanced extension is **block bootstrap over whole sprint observations**. In the simplest form, that means:

- resample whole sprint rows at the configured cadence whenever possible;

- when history spans multiple cadences or adjacent operational regimes, resample cadence-specific blocks of whole sprint rows rather than synthetic week fragments.

That approach better preserves ceremony effects, batching, within-sprint coupling, and end-of-sprint completion behavior.

2.4 Joint Empirical Bootstrap

The default stochastic engine should use **joint bootstrap resampling**. In practical terms, that means drawing historical outcome vectors with replacement from the normalized historical data, not sampling each component independently.

If the normalized historical dataset has n usable rows, define:

$$Y_i = (w_i^c, w_i^s, w_i^a, w_i^r)$$

for the MVP fallback path, or equivalently:

$$Z_i = (\hat{c}_i, \hat{s}_i, \hat{a}_i, \hat{r}_i, L_i)$$

for the preferred sprint-entity resampling path.

For the MVP weekly fallback, for each simulated future week t, draw an index:

$$J_t \sim \text{Uniform}(\{1, 2, \ldots, n\})$$

and use:

$$(W_t^c, W_t^s, W_t^a, W_t^r) = Y_{J_t}$$

For the preferred sprint-entity path, draw a historical sprint row or block of rows that already matches the configured cadence and use that as the sampled future sprint outcome.

This is the core statistical choice in the proposal. It preserves the observed dependence structure between good sprints, churn-heavy sprints, and low-delivery sprints. If high added scope historically coincides with high spillover and lower completion, the forecast should preserve that relationship instead of averaging it away. The main refinement is that, when possible, the preserved dependence structure should be attached to whole sprint entities rather than reconstructed from synthetic week fragments.

2.5 Derived Historical Ratios

The model should compute three derived ratios from the same historical rows for diagnostics and planned-load guidance:

$$\text{spillover_ratio}_i = \frac{s_i}{\max(c_i + s_i, \epsilon)}, \quad \text{scope_addition_ratio}_i = \frac{a_i}{\max(c_i + a_i, \epsilon)}, \quad \text{scope_removal_ratio}$$

These ratios summarize different aspects of instability:

- spillover ratio measures the share of attempted sprint scope that did not land.
- scope addition ratio measures how much demand arrived after planning.
- scope removal ratio measures how much planned scope had to be taken back out.

The proposal uses these ratios in two places: for reporting historical planning stability, and for computing conservative planned-load guidance.

2.6 Planned-Load Guidance Heuristic

The proposal deliberately separates the **forecasting engine** from the **commitment heuristic**.

- The completion forecast comes from full Monte Carlo simulation over backlog state.
- The planned-load recommendation is a simpler percentile-based heuristic for near-term sprint planning.

Let $Q_p(X)$ denote the empirical percentile or empirical quantile of series X at probability level p. Then for planning confidence level q, the recommended commitment is:

$$\text{recommended_planned_commitment}(q) = \max\left(0, Q_{50}(\text{completed_units}) \cdot (1 - Q_q(\text{spillover_ratio})) \cdot\right.$$

This formula is intentionally conservative. It starts from typical delivered capacity, discounts it by high-percentile spillover and removal behavior, and reserves room for likely urgent additions. It is a planning aid, not the main forecast generator.

2.7 Backlog-State Recursion

Completion-date forecasting should be based on explicit backlog evolution rather than on dividing remaining work by a point estimate. The per-sprint backlog recursion is:

$$B_{t+1} = B_t - D_t + A_t - R_t^{\text{effective}}$$

where:

- B_t = remaining backlog at the start of sprint t
- D_t = actually delivered units in sprint t
- A_t = sampled added work in sprint t
- $R_t^{\text{effective}}$ = sampled removed work that counts as genuine backlog reduction

The removed work treatment flag determines $R_t^{\text{effective}}$:

- in churn only mode, $R_t^{\text{effective}} = 0$
- in reduce backlog mode, $R_t^{\text{effective}} = R_t$

This recursion is the main mechanism that turns historical churn into a date forecast.

2.8 Volatility Overlay

The optional volatility overlay should be modeled as a multiplicative factor on deliverable capacity, not as a rewrite of the historical churn series:

$$C_t^{\text{effective}} = C_t^{\text{sampled}} \cdot V_t$$

where $V_t = 1.0$ when volatility is disabled and otherwise comes from a configured disruption model. This preserves a clear statistical separation:

- empirical history captures normal observed variation
- volatility overlay captures extra exogenous disruption
- churn remains churn

Future sprint overrides such as public-holiday sprints are then applied as explicit forward-looking capacity multipliers:

$$C_t^{\text{final}} = C_t^{\text{effective}} \cdot O_t$$

where O_t defaults to 1.0 and is derived from future sprint overrides when present.

2.9 Task-Level Execution Spillover

Task-level execution spillover is a second stochastic layer that applies after sprint capacity is sampled and tasks are pulled into the sprint. It is intentionally distinct from historical sprint-row spillover.

For a task with planning size s, the default spillover probability is either:

$$P(\text{spillover} \mid s) = \frac{1}{1 + \exp(-(a \log(s/\text{ref_size}) + b))}$$

for the logistic model, or a piecewise table over configured size brackets for the default explainable model.

If a spillover event occurs, the consumed fraction is sampled from a Beta distribution:

$$F \sim \text{Beta}(\alpha, \beta), \quad \text{remaining_effort} = \text{planned_effort} \cdot (1 - F)$$

with starter defaults $\alpha = 3.25$ and $\beta = 1.75$, giving an expected consumed fraction of:

$$E[F] = \frac{\alpha}{\alpha + \beta} = \frac{3.25}{5.0} = 0.65$$

This means most of the task effort is usually consumed before the task spills over, but the task still contributes zero delivered throughput in that sprint.

2.10 Output Statistics

The forecast should report two families of statistics:

1. **Historical input diagnostics** computed from the normalized history and derived ratios.
2. **Simulated output diagnostics** computed from the Monte Carlo sprint-count and date distributions.

The primary summary outputs are empirical percentiles such as P50, P80, and P90. The proposal also uses standard descriptive statistics such as mean, median, standard deviation, and coefficient of variation. Where the proposal refers to "correlation statistics," the intended default is the Pearson correlation coefficient computed over the historical completed, spillover, added, and removed series, with Spearman rank correlation permitted as an additional optional diagnostic.

2.11 Why These Methods Fit `mcprojsim`

These methods are intentionally aligned with the current repository architecture:

- they preserve Monte Carlo sampling as the central forecasting mechanism;
- they respect the existing distinction between elapsed duration and total effort;
- they avoid forcing a deterministic burn-rate model into a codebase that already uses repeated stochastic simulation;
- they keep sprint planning as a parallel forecast surface rather than trying to reinterpret the current duration arrays.

Chapter 3

Natural Language Input and MCP Integration

The sprint-planning extension should also be expressible through the repository's existing semi-structured natural-language path, because that is already how the MCP server generates project files from text through `src/mcprojsim/nl_parser.py`. The design should therefore define a simplified natural-language surface for sprint planning that maps directly onto the same canonical `sprint_planning` schema used by YAML and TOML input.

This should remain **semi-structured natural language**, not unconstrained free prose. The current parser works best when users provide short labeled lines, headers, and bullets. Sprint-planning support should follow that same contract so MCP users can describe planning settings and history in a way that still parses deterministically.

3.1 Suggested Natural-Language Shape

The cleanest extension is to allow an optional sprint-planning section and repeated historical sprint sections inside the natural-language project description.

Suggested shape:

```
Project name: Payments Platform Refresh
Start date: 2026-04-06

Sprint planning:
- Enabled: yes
- Sprint length: 2 weeks
- Capacity mode: story points
- Planning confidence: 80%
- Removed work treatment: churn only

Sprint history SPR-2026-01:
- Completed: 21 story points
- Spillover: 5 story points
- Added: 3 story points
- Removed: 1 story point
- Holiday factor: 1.0

Sprint history SPR-2026-02:
- Completed: 18 story points
- Spillover: 7 story points
- Added: 6 story points
- Removed: 2 story points
- Holiday factor: 0.9
```

The natural-language parser should normalize that form into the canonical YAML structure:

- `Sprint planning:` starts a planning block;
- `Sprint history <id>:` starts one historical sprint row;
- the parsed output should still be emitted as the same `sprint_planning` YAML block described elsewhere in this proposal.

3.2 Accepted Human Variants

Because the purpose of this input mode is to reduce friction for MCP and text-first users, the parser should accept common human alternatives for the same underlying fields, then normalize them to canonical values.

Recommended alias handling:

- sprint length: `Sprint length: 2 weeks`, `2 week sprints`, `Sprint cadence = 2 weeks`
- capacity mode: `story points`, `points`, `tasks`, `items`
- planning confidence: `Planning confidence: 80%`, `Plan at 80 percent confidence`, `Confidence level = 0.80`
- removed-work treatment: `churn only`, `count removed work as churn only`, `reduce backlog`, `descoped work reduces backlog`
- completed work: `completed`, `finished`, `done`, `delivered`
- spillover work: `spillover`, `carryover`, `rolled over`

- added work: `added`, `scope added`, `unplanned work added`, `mid-sprint additions`
- removed work: `removed`, `descoped`, `taken out`, `scope removed`

To keep the grammar deterministic, the parser should not try to infer sprint-planning semantics from arbitrary paragraphs. The accepted synonyms should still appear in short labeled lines or bullets.

3.3 Natural-Language Example

The following example shows the intended user-facing input style, including several common human variations of the same concepts:

```
Project name: Payments Platform Refresh
Start date: 2026-04-06
Description: Refresh the payments platform and move service integrations in stages.

Sprint planning:
- Turn sprint planning on
- We work in 2-week sprints
- Plan using points
- Plan at 80 percent confidence
- Removed work counts as churn only

Sprint history SPR-2026-01:
- Done: 21 points
- Carryover: 5 points
- Added mid-sprint: 3 points
- Descoped: 1 point
- Holiday factor: 1.0
- Notes: Normal sprint

Sprint history SPR-2026-02:
- Finished: 18 story points
- Rolled over: 7 story points
- Scope added: 6 story points
- Taken out: 2 story points
- Capacity was 90 percent because of holidays

Sprint history SPR-2026-03:
- Delivered: 24 points
- Spillover: 2 points
- Unplanned work added: 1 point
- Removed: 0 points

Task 1: Discovery and architecture
- Story points: 5

Task 2: Schema updates
- Story points: 8
- Depends on Task 1

Task 3: API implementation
- Story points: 8
- Depends on Task 2
```

That example deliberately mixes `done`, `finished`, and `delivered`, as well as `carryover`, `rolled over`, and `spillover`. The parser should normalize all of them to the canonical historical fields. Likewise, phrases such as `Capacity was 90 percent because of holidays` should normalize to `holiday_factor: 0.9` when the intent is clear and the numeric value is explicit.

3.4 Scope of the Natural-Language Form

The simplified natural-language form only needs to cover the sprint-planning fields that are realistic for text-first authoring and MCP interaction:

- `enabled`
- `sprint_length_weeks`
- `capacity_mode`
- `planning_confidence_level`
- `removed_work_treatment`
- inline historical sprint rows with `sprint_id`, completed work, spillover, added work, removed work, `holiday_factor`, and optional notes

Future sprint overrides, volatility overlays, and other advanced structures can remain YAML/TOML-first in the MVP and be added to the natural-language form later if there is real usage pressure.

Chapter 4

Formal Requirements

The requirements below are grouped by configuration, simulation behavior, outputs, validation, and implementation structure. They define the full target-state specification for sprint planning. The roadmap above defines implementation sequencing and MVP boundaries; it does not weaken any requirement outside the scope of the current increment. `SHALL` denotes mandatory behavior for whatever increment claims that requirement in scope. `MAY` denotes explicitly permitted optional behavior.

4.1 Configuration Requirements

4.1.1 FR-SP-001: Sprint Planning Mode Integration

- The system SHALL support sprint-based planning as an additional, parallel planning mode that does not replace or alter the existing Monte Carlo duration/effort simulation.
- Sprint planning SHALL be enabled via a dedicated top-level `sprint_planning` section in the project definition file.
- Existing project files that omit the `sprint_planning` section SHALL remain fully valid and SHALL run in the existing simulation mode without change.

4.1.2 FR-SP-002: Sprint Length Configuration

- The system SHALL allow the sprint length to be configured as a whole number of weeks.
- The sprint length SHALL be applied consistently to all sprints within a simulation run.
- Historical sprint outcome entries that use a different sprint length than the configured sprint length SHALL be normalized to a weekly outcome rate before resampling.

4.1.3 FR-SP-003: Capacity Planning Units

- The system SHALL support two mutually exclusive capacity modes: `story_points` and `tasks`.
- In `story_points` mode, each task SHALL expose a planning story-point value, either via an

existing story-point estimate or a separate `planning_story_points` field on the task, and sprint capacity SHALL be measured in story points.

- In `tasks` mode, each eligible task SHALL count as one unit of work, and sprint capacity SHALL be measured in completed task count.
- The system SHALL warn when `tasks` mode is used and task sizes are heterogeneous, as throughput-based forecasting is only reliable when items are roughly comparable in size.
- The system SHALL NOT silently convert hours-based duration estimates into story points.
- The system SHALL treat task-count capacity and task execution size as separate concepts: `tasks` mode uses task count for sprint-capacity accounting, while optional execution-spillover behavior MAY use a task's explicit planning story-point size when such a size is available.

4.1.4 FR-SP-004: Historical Sprint Outcome Input

- The system SHALL accept a list of historical sprint outcome observations provided by the user in the project definition file.
- The system SHALL also support a simplified semi-structured natural-language representation of sprint-planning input for use by `src/mcprojsim/nl_parser.py` and the MCP server tools that already depend on that parser.
- The natural-language representation SHALL normalize accepted aliases and phrasing variants into the canonical `sprint_planning` schema before YAML generation, validation, or simulation.
- After the MVP increment, the system SHALL also support an alternative history input mode in which `sprint_planning.history` references an external JSON or CSV file instead of embedding inline history rows.
- The system SHALL allow exactly one history source mechanism per sprint-planning configuration: either inline rows or an external history file reference.
- When an external history file is used, the path SHALL be resolved relative to the project file unless an absolute path is provided.
- External history rows SHALL be parsed into the same canonical row structure and SHALL then follow the same validation, defaulting, normalization, and statistical processing rules as inline rows.
- Every historical sprint outcome entry SHALL include a `sprint_id` field that serves as the primary key for that historical sprint record.
- Every historical sprint outcome entry SHALL include exactly one of `completed_story_points` or `completed_tasks`.
- Any historical sprint outcome entry field other than `sprint_id` and the required completed-unit field MAY be omitted.
- In `story_points` mode, a history entry MAY provide `completed_story_points`, `spillover_story_po` `added_story_points`, and `removed_story_points`.
- In `tasks` mode, a history entry MAY provide `completed_tasks`, `spillover_tasks`, `added_tasks`, and `removed_tasks`.
- In external JSON history files, each row SHALL use the same canonical field names as the inline history schema.

- In external CSV history files, the header row SHALL use the same canonical field names as the inline history schema.
- When `completed_story_points` is used in a history entry, any spillover, added, and removed values for that entry SHALL use the corresponding `*_story_points` fields.
- When `completed_tasks` is used in a history entry, any spillover, added, and removed values for that entry SHALL use the corresponding `*_tasks` fields.
- The system SHALL treat `end_date` as optional metadata and SHALL NOT require it in order to identify or store a historical sprint row.
- The system SHALL require at least two usable historical observations with positive delivery signal after defaulting and normalization before running a sprint simulation.

4.1.5 FR-SP-005: Sprint Planning Defaults

- When `sprint_length_weeks` is omitted from a history entry, the system SHALL default it to the parent `sprint_planning.sprint_length_weeks`.
- When `completed_*`, `spillover_*`, `added_*`, or `removed_*` fields are omitted, the system SHALL treat the missing value as `0` for that sprint.
- When `holiday_factor` is omitted, the system SHALL treat it as `1.0` for that sprint.
- When `end_date`, `team_size`, or `notes` are omitted, the system SHALL treat them as null/ignored metadata values.
- When `removed_work_treatment` is omitted, the system SHALL default it to `churn_only`.
- When `planning_confidence_level` is omitted, the system SHALL default it to the documented project default, proposed here as `0.80`.

4.1.6 FR-SP-006: Empirical Joint Outcome Resampling

- The default sprint-capacity model SHALL be empirical bootstrap resampling from the provided historical sprint outcome observations.
- The system SHALL resample completed units, spillover units, added units, and removed units as a joint historical vector so that observed correlations between delivery, spillover, and scope churn are preserved.
- The system SHALL support historical rows where any fields are absent by first normalizing all omitted fields to their neutral defaults before statistical processing.
- The system SHALL distinguish delivery-capacity signals from churn signals when normalizing historical data, so that holiday/calendar adjustments do not directly rescale raw added-work or removed-work observations.
- The system SHALL treat the historical sprint as the primary resampling entity whenever the configured cadence is represented in the available history.
- For mixed historical sprint lengths or requested cadences not represented in history, the MVP MAY fall back to per-week outcome-rate normalization and re-aggregation as an approximation.
- The system MAY support a more advanced block-bootstrap extension over whole sprint observations or cadence-specific blocks as the preferred alternative to synthetic week-by-week

recombination.
- The system SHALL NOT default to using only the historical mean velocity as the sprint capacity; point-estimate forecasting SHALL NOT be the primary output.

4.1.7 FR-SP-007: Historical Spillover Modeling

- The system SHALL treat historical spillover as a first-class stochastic input rather than only as explanatory metadata.
- The system SHALL derive a spillover ratio from the historical data and use it to quantify planning instability.
- The system SHALL use historical spillover information to reduce recommended planned sprint load and to calibrate future sprint spillover behavior in simulation.

4.1.8 FR-SP-008: Historical Scope-Addition Modeling

- The system SHALL treat historical added work as a first-class stochastic input rather than only as explanatory metadata.
- The system SHALL use historical added-work observations to model future unplanned scope growth during simulation.
- The system SHALL use historical added-work observations to reserve part of future sprint capacity for expected urgent work that arrives after sprint start when computing recommended planned sprint-load guidance.
- The system SHALL NOT implement that reservation as a second independent reduction of simulated sprint capacity beyond the explicit sampled added-work term already applied in the sprint simulation.

4.1.9 FR-SP-009: Historical Scope-Removal Modeling

- The system SHALL treat historical removed work as a first-class stochastic input rather than only as explanatory metadata.
- The system SHALL derive a scope-removal ratio from the historical data and use it to quantify planning churn.
- The system SHALL use historical removed-work observations to reduce recommended planned sprint load when frequent de-scoping indicates unstable sprint commitments.
- The system SHALL support an explicit configuration choice to treat removed work either as churn-only signal or as effective backlog reduction in forecast simulations.

4.1.10 FR-SP-010: Historical Metadata Treatment

- The system SHALL treat `end_date`, `team_size`, and `notes` as metadata-only fields in the MVP sprint forecast unless a later feature explicitly enables them as model inputs.
- The system SHALL treat `sprint_id` as the mandatory key for storing and referring to each historical sprint row, not as optional metadata.

- Metadata-only fields MAY be preserved for reporting, ordering, filtering, or future analysis, but SHALL NOT silently change sprint-capacity calculations in the MVP design.

4.2 Simulation Behavior Requirements

4.2.1 FR-SP-011: Dependency-Aware Sprint Pull Simulation

- The system SHALL maintain a ready queue of tasks whose declared dependencies have been satisfied in prior sprints.
- In each simulated sprint, the system SHALL pull tasks from the ready queue in priority order until the sampled sprint capacity would be exceeded.
- When a priority field is present, the planner SHALL pull lower numeric priorities first and break ties by task ID.
- When no priority field is present, the planner SHALL use a deterministic stable ordering based on task ID.
- The system SHALL inject sampled unplanned work into the sprint and backlog according to the historical added-work model.
- The system SHALL remove sampled de-scoped work from the sprint and optionally from the remaining backlog according to the configured removed-work treatment.
- A task that does not fit within remaining sprint capacity SHALL be deferred to a future sprint without consuming any capacity.
- Only fully completed tasks SHALL be credited to delivered throughput for the sprint.
- After each sprint, the system SHALL unlock tasks whose dependencies are now fully satisfied and add them to the ready queue.
- The system SHALL repeat the sprint cycle until all tasks in the project backlog have been completed.
- The system SHALL make the backlog update rule explicit and auditable per iteration so that delivery, added scope, removed scope, capacity-driven deferral, and execution-driven spillover can be distinguished in diagnostics and exports.
- When historical added work or removed work is sampled as aggregate units rather than as named tasks, the system SHALL represent those changes explicitly as auditable synthetic backlog adjustments or equivalent ledger entries rather than silently folding them into capacity.
- Each simulation iteration SHALL record the total number of sprints required and the projected completion date.

4.2.2 FR-SP-012: Future Sprint Override Configuration

- The system SHALL support explicit configuration of future sprint-specific capacity overrides for known calendar or availability events.
- A future sprint override SHALL allow identifying the target sprint by at least one explicit locator such as sprint number or start date.
- A future sprint override MAY specify a `holiday_factor`, `capacity_multiplier`, or equivalent explicit availability reduction.

- When a future sprint override provides multiple locators, those locators SHALL resolve to the same simulated sprint; otherwise validation SHALL fail.
- When a future sprint override matches a simulated future sprint, the system SHALL apply that override to the effective capacity of the targeted sprint.
- When both `holiday_factor` and `capacity_multiplier` are provided on the same override, the system SHALL apply both multiplicatively.
- When no future sprint override is configured for a sprint, the forecast SHALL assume a neutral future override of `1.0`.

4.2.3 FR-SP-013: Volatility Overlay

- The system SHALL support an optional sprint-level capacity volatility overlay that is disabled by default.
- When enabled, the system SHALL apply a multiplicative disruption factor to sampled sprint capacity with a configurable disruption probability and a configurable impact range.
- The effective sprint capacity SHALL be computed as the product of the resampled historical completed-capacity component and the sampled disruption multiplier.
- When no volatility overlay is configured, the multiplier SHALL default to 1.0, preserving the empirical-only behavior.
- The system SHALL support scenario-driven capacity overrides for individual future sprints to model known planned events such as public holidays or team unavailability.
- The volatility overlay SHALL affect deliverable sprint capacity and SHALL NOT directly rescale raw historical added-work or removed-work observations.
- The volatility overlay SHALL remain distinct from task-level spillover and from scope-churn variables, which continue to be modeled separately.

4.2.4 FR-SP-014: Task-Level Execution Spillover Modeling

- The system SHALL support an optional task-level spillover model that is disabled by default.
- When enabled, each task pulled into a sprint SHALL be evaluated for execution spillover using a size-dependent probability model.
- The probability of spillover SHALL increase monotonically with planned task size so that larger tasks have a materially higher probability of spilling over than smaller tasks.
- The system SHALL support two spillover probability models: a table-based piecewise model (default) and a logistic function of log task size.
- The default table-based model SHALL assign spillover probabilities of 0.05 for tasks of 1–2 SP, 0.12 for 3–5 SP, 0.25 for 6–8 SP, and 0.40 for tasks larger than 8 SP.
- The probability brackets SHALL be configurable in the project definition file.
- The spillover probability model SHALL require a resolvable planning size expressed in story points for each task it evaluates.
- When `capacity_mode` is `tasks`, the spillover model SHALL use each task's explicit planning story-point size if available; otherwise the system SHALL reject that configuration or require the spillover model to be disabled.

- Individual tasks MAY declare a `spillover_probability_override` to supersede the size-bracket default.

4.2.5 FR-SP-015: Spillover Effort Carryover

- When a spillover event is triggered for a task, the system SHALL sample the fraction of planned effort consumed during the current sprint from a Beta distribution.
- The default Beta distribution parameters SHALL yield a mean consumed fraction of approximately 0.65 (`alpha = 3.25`, `beta = 1.75`).
- The consumed fraction SHALL be charged against the current sprint's capacity budget in the same unit used by the spillover sizing model.
- The task SHALL NOT be credited to delivered throughput in the current sprint.
- The remaining effort SHALL be re-entered into the ready queue as a reduced-size remainder task for the next sprint, retaining the original task's dependency relationships (which are already satisfied at the point of spillover).
- The Beta distribution parameters SHALL be configurable in the project definition file.

4.3 Output and Reporting Requirements

4.3.1 FR-SP-016: Sprint Simulation Output and Statistics

- The system SHALL run N iterations of the sprint simulation, where N uses the same configurable iteration count as the existing Monte Carlo engine.
- The system SHALL report P50, P80, and P90 percentiles for the number of sprints required to complete the project.
- When a project start date is provided, the system SHALL map simulated sprint counts to projected delivery dates using sprint boundaries rather than working-day accumulation.
- The system SHALL report projected delivery dates corresponding to each reported confidence level when a project start date is provided.
- The system SHALL report recommended planned sprint-load guidance for at least one planning confidence level, using the configured value or the documented default when the field is omitted.
- The system SHALL report descriptive statistics for the simulated sprint-count distribution, including mean, median, standard deviation, and coefficient of variation.
- The system SHALL report descriptive statistics for the input historical completed-work, spillover, added-work, and removed-work series, including mean, median, standard deviation, and coefficient of variation.
- The system SHALL report spillover ratio, scope-addition ratio, and scope-removal ratio percentiles derived from the historical sprint outcome data.
- The system SHALL report Pearson correlation coefficients between historical completed, spillover, added, and removed units so the joint outcome structure remains visible to users.
- The system SHALL support burn-up style percentile bands for simulated sprint progress across future sprints as an exporter-ready data structure containing, for each forecast sprint index,

percentile values for cumulative delivered units at least at P50, P80, and P90.

4.3.2 FR-SP-017: Carryover and Spillover Diagnostics

- The system SHALL distinguish between capacity-driven deferral (task not started, no capacity consumed) and execution-driven spillover (task started, capacity partially consumed, not delivered) in all diagnostic output.
- The system SHALL report, per simulation run, the distribution of carryover load across sprints, including mean and P80/P90 carryover effort.
- The system SHALL report the aggregate spillover rate as the fraction of task-sprint assignments that resulted in a spillover event.

4.3.3 FR-SP-018: Export and Reporting

- Sprint planning results SHALL be included in JSON and CSV exports when the sprint planning mode is active.
- Sprint planning results SHALL be clearly separated from the existing duration/effort simulation results in all output formats.
- CLI output SHALL indicate whether sprint planning mode was active and summarize the P50/P80/P90 sprint-count results.
- Exports SHALL include the completed-work, spillover, added-work, and removed-work statistics for the input historical series alongside the simulated sprint-count percentiles.
- Exports SHALL include the recommended planned sprint-load guidance and the planning confidence level used to compute it.
- When the volatility overlay is enabled, exports SHALL report the disruption probability and the simulated disruption frequency observed across iterations.
- When the spillover model is enabled, exports SHALL include the aggregate spillover rate and the carryover distribution summary.
- Exports SHALL include the historical spillover ratio, scope-addition ratio, and scope-removal ratio summaries used in the forecast.
- Exports SHALL include the historical correlation summaries used to preserve the joint outcome interpretation of completed, spillover, added, and removed work as a fixed pairwise structure keyed by metric pair and containing at minimum the Pearson correlation coefficient for each pair.
- Exports SHALL include burn-up style percentile bands, when those outputs are generated, as per-sprint percentile series rather than only as pre-rendered charts.
- Exports SHALL include the configured removed-work treatment used when projecting remaining backlog and completion dates.

4.4 Validation Requirements

4.4.1 FR-SP-019: Schema and Validation

- The system SHALL validate that `sprint_length_weeks` is a positive integer.
- The system SHALL validate that `capacity_mode` is one of the supported enumerated values.
- The system SHALL validate that every historical sprint outcome entry includes a non-empty `sprint_id`.
- The system SHALL validate that `sprint_id` values are unique within the provided historical sprint series.
- The system SHALL validate that `sprint_planning.history` is specified as either inline rows or an external history source descriptor, but not both.
- The system SHALL validate that external history sources declare a supported format of `json` or `csv` and a non-empty path.
- The system SHALL validate that every historical sprint outcome entry includes exactly one of `completed_story_points` or `completed_tasks`.
- The system SHALL validate that all historical `completed_*`, `spillover_*`, `added_*`, and `removed_*` values are non-negative.
- The system SHALL validate that every historical `holiday_factor`, when provided, is strictly positive.
- The system SHALL validate that every historical row can be normalized by applying neutral defaults to omitted fields before downstream statistical processing.
- The system SHALL validate that the history contains at least two usable observations with positive delivery signal after defaulting and normalization.
- The system SHALL validate that external JSON history files contain an array of row objects and that external CSV history files contain a header row with canonical field names.
- The system SHALL validate that each history entry uses the unit family implied by `capacity_mode` and does not mix task counts with story-point values for the same simulation.
- The system SHALL validate that `planning_confidence_level`, when provided, is in the open interval `(0, 1)`.
- The system SHALL validate that `removed_work_treatment`, when provided, is one of `churn_only` or `reduce_backlog`.
- The system SHALL validate that any configured future sprint override targets a uniquely identifiable sprint and uses positive multiplier values.
- The system SHALL validate that any future sprint override containing both sprint number and start date locators resolves those locators to the same target sprint.
- The system SHALL validate that any configured future-sprint `holiday_factor`, when provided, is strictly positive.
- The system SHALL validate that all spillover probability values are in the range [0.0, 1.0].
- The system SHALL validate that the Beta distribution parameters `alpha` and `beta` are both strictly positive.
- Tasks referencing `spillover_probability_override` SHALL fail validation if the value is outside [0.0, 1.0].

- When `capacity_mode` is `story_points`, the system SHALL validate that every task in the project backlog has a resolvable planning story-point value and SHALL report a validation error for any task that does not.
- When task-level spillover modeling is enabled, the system SHALL validate that every task eligible for spillover evaluation has a resolvable planning story-point size and SHALL fail validation otherwise.

4.5 Implementation Structure Requirements

4.5.1 FR-SP-020: Internal Component Structure

- Sprint planning logic SHALL be implemented in a dedicated `planning/` module to preserve separation of concerns and avoid modifying the existing simulation engine.
- The module SHALL contain at minimum: a `SprintCapacitySampler` for resampling historical sprint outcome vectors, a `SprintPlanner` for managing the ready queue and sprint pulling, a `SprintSimulationEngine` for running N iterations, and a `SprintPlanningResults` model for holding output arrays and statistics.
- The sprint simulation engine SHALL reuse the existing project's random seed mechanism to ensure reproducible results when a seed is configured.
- After the MVP increment, the parsing layer SHALL include a dedicated history-file parser component for external JSON and CSV sprint-history sources so those files are normalized into `SprintHistoryEntry` rows before statistical processing.

Chapter 5

Implementation Work Breakdown

The current code base already has clean boundaries for configuration, raw-file validation, Pydantic schema validation, Monte Carlo execution, results modeling, CLI presentation, and exporters. Sprint planning should be implemented as a parallel path through those same boundaries rather than by overloading the existing duration engine.

5.1 Extend the Project Schema First

Start in the schema layer because all downstream work depends on stable validated models.

Primary files:

- `src/mcprojsim/models/project.py`
- `src/mcprojsim/config.py`

Steps:

1. Add a new optional top-level `sprint_planning` field to `Project` in `src/mcprojsim/models/project.py`
2. Introduce new Pydantic models for `SprintPlanningSpec`, `SprintHistoryEntry`, `FutureSprintOverri` `SprintVolatilitySpec`, and `SprintSpilloverSpec` in the schema layer.
3. Add task-level fields needed by the design, most likely `planning_story_points`, `priority`, and `spillover_probability_override`, on the existing `Task` model.
4. Keep duration-estimation fields and sprint-planning fields separate. The existing `TaskEstimate.st` currently maps symbolic estimates to time via config, so sprint planning should not overload that field with backlog-planning semantics.
5. Add project-level defaults that belong in configuration, such as the documented planning confidence default, to `src/mcprojsim/config.py` rather than hardcoding them in multiple runtime components.
6. Extend `SprintPlanningSpec.history` so it can also hold a `SprintHistorySourceSpec` with `format` and `path` for external CSV or JSON history files.

Verification:

- `Project(**data)` still loads a file with no `sprint_planning` section unchanged.
- `Project(**data)` also loads a fully specified sprint-planning block with all optional sections present.

5.2 Add Raw Validation and Source-Aware Error Reporting

The repository already performs useful path-aware validation before Pydantic model construction. Sprint planning should plug into that same layer so YAML and TOML errors retain line context.

Primary files:

- `src/mcprojsim/parsers/error_reporting.py`
- `src/mcprojsim/parsers/yaml_parser.py`
- `src/mcprojsim/parsers/toml_parser.py`

Steps:

1. Extend `validate_project_payload()` in `src/mcprojsim/parsers/error_reporting.py` with sprint-planning-specific raw-data checks that benefit from direct source paths.
2. Add checks for duplicate `sprint_id` values, mixed unit families inside history rows, invalid override locators, and malformed spillover bracket definitions.
3. Keep deeper semantic validation in the Pydantic models, but use the raw validator for issues where a source-aware message is materially better.
4. Preserve backward compatibility for existing project files and parser behavior.

Post-MVP extension:

5. Add a dedicated parser for external sprint-history files, for example `src/mcprojsim/parsers/sprint` that supports both JSON arrays of row objects and CSV files with canonical column names.

Verification:

- YAML and TOML files with bad sprint-planning data produce line-aware validation messages.
- Existing non-sprint project files validate exactly as before.

5.3 Implement the New Planning Engine as a Parallel Subsystem

Do not fold sprint logic into `SimulationEngine.run()`. The current duration engine in `src/mcprojsim/sim` should remain stable.

Primary files:

- new `src/mcprojsim/planning/sprint_capacity.py`
- new `src/mcprojsim/planning/sprint_planner.py`
- new `src/mcprojsim/planning/sprint_engine.py`
- existing `src/mcprojsim/simulation/engine.py` for seed-reuse patterns only
- existing `src/mcprojsim/simulation/scheduler.py` as a dependency-order reference

Steps:

1. Implement `SprintCapacitySampler` to normalize sparse historical rows, holiday-normalize delivery-side quantities, resample whole sprint observations as the default entity when cadence matches are available, use weekly normalization only as a mixed-cadence fallback, perform joint bootstrap resampling, and apply volatility overlay plus future sprint overrides.
2. Implement `SprintPlanner` to build the dependency-ready queue from existing task dependencies, apply deterministic pull order using priority then task ID, defer tasks that do not fit, account for execution spillover when enabled, and record explicit backlog adjustments for aggregate added/removed work.
3. Implement `SprintSimulationEngine` to run iterations with the same random-seed semantics already used by `SimulationEngine` in `src/mcprojsim/simulation/engine.py`.
4. Reuse ideas from `TaskScheduler` in `src/mcprojsim/simulation/scheduler.py` for deterministic dependency handling, but do not try to reuse the elapsed-time scheduler directly because the sprint planner is a different abstraction.

Verification:

- identical seed plus identical input produces identical sprint-planning outputs;
- the new engine runs independently of the existing duration engine;
- disabling sprint planning leaves the current simulation path untouched.

5.4 Add a Dedicated Sprint Results Model

The current `SimulationResults` model in `src/mcprojsim/models/simulation.py` is duration-centric. Sprint planning should produce a separate results surface instead of mutating that class into a mixed abstraction.

Primary files:

- new `src/mcprojsim/models/sprint_simulation.py` or equivalent
- `src/mcprojsim/models/simulation.py` only if a thin composition hook is needed

Steps:

1. Add a `SprintPlanningResults` model holding sprint-count arrays, date percentiles, historical diagnostics, carryover statistics, planned-load guidance, and burn-up percentile data.
2. Mirror the existing result-model style where useful: percentile helpers, dictionary/export helpers, and summary-statistics calculation.
3. Keep sprint outputs structurally separate from `SimulationResults` so exports and CLI output can present both views without conflating elapsed time, effort, and sprint count.

Verification:

- sprint-specific statistics can be computed without depending on `SimulationResults.durations`.
- results serialization stays straightforward for JSON, CSV, and HTML exporters.

5.5 Integrate the CLI Without Breaking the Current Command Model

The existing entry point is the `simulate` command in `src/mcprojsim/cli.py`. Sprint planning should be added there as an extra output path, not as a new requirement for all simulations.

Primary files:

- `src/mcprojsim/cli.py`

Steps:

1. Detect whether `project.sprint_planning` is present and enabled after parsing the project file.
2. Run the current duration simulation exactly as today.
3. Run sprint planning as an additional analysis step when enabled.
4. Extend CLI output to clearly label sprint-planning mode, report sprint-count percentiles, date percentiles, commitment guidance, and the selected `removed_work_treatment`.
5. Keep quiet and minimal modes compatible with the new output surface.

Verification:

- existing CLI workflows still work for projects without sprint planning;
- sprint-enabled projects produce both legacy simulation output and sprint-planning output in a clearly separated form.

5.6 Extend Exporters with a Parallel Sprint-Planning Section

The exporter layer is already cleanly separated by format. Sprint planning should be added as a new section in each exporter rather than mixed into existing duration statistics.

Primary files:

- `src/mcprojsim/exporters/json_exporter.py`
- `src/mcprojsim/exporters/csv_exporter.py`
- `src/mcprojsim/exporters/html_exporter.py`

Steps:

1. Add a dedicated sprint-planning section to JSON export rather than flattening sprint metrics into the existing top-level statistics block.
2. Add CSV sections for sprint-count percentiles, historical diagnostics, carryover diagnostics, and planned-load guidance.
3. Extend the HTML report with a clearly separated sprint-planning area that can later absorb burn-up percentile charts.
4. Ensure exports include the diagnostic distinctions required by the spec: completed vs added vs removed vs historical spillover vs carryover.

Verification:

- exporters still work for pure duration simulations;
- sprint-enabled exports contain the new sections and remain parseable.

5.7 Add Tests in the Same Layers the Repository Already Uses

The repository already has a well-distributed test layout. Sprint planning should follow that pattern instead of creating only end-to-end tests.

Primary files to extend:

- `tests/test_models.py`
- `tests/test_parsers.py`
- `tests/test_simulation.py`
- `tests/test_results.py`
- `tests/test_cli.py`
- `tests/test_exporters.py`
- possibly `tests/test_integration.py` or `tests/test_e2e_combinations.py`

Steps:

1. Add schema-validation tests for sparse history rows, duplicate `sprint_id`, mixed unit families, and invalid spillover configuration.
2. Add sampler tests for holiday normalization, whole-sprint entity resampling, weekly-normalization fallback behavior, and joint resampling behavior.
3. Add planner tests for deterministic pull order, deferral, execution spillover, and backlog-adjustment ledger behavior.
4. Add results and export tests for sprint-count percentiles, planned-load guidance, and carryover diagnostics.
5. Add CLI integration tests showing that sprint planning is optional and non-breaking.

Verification:

- a minimal sprint-planning project file passes end to end;
- invalid files fail with precise messages;
- current non-sprint tests remain green.

5.8 Recommended Implementation Order

The shortest safe delivery sequence on the current code base is:

1. schema and validation
2. sprint results model
3. capacity sampler
4. sprint planner
5. sprint simulation engine
6. CLI integration

7. exporters
8. burn-up and advanced reporting
9. external history file import for CSV and JSON

That order works because each layer depends on the previous one and because it keeps the existing duration simulation path stable until sprint planning is already functionally complete.

Chapter 6

MVP-Only Implementation Plan

This section extracts the first increment only from the broader implementation work breakdown. It is intentionally limited to the roadmap MVP scope:

- `sprint_planning` project schema
- `capacity_mode = story_points | tasks`
- historical `spillover_*`, `added_*`, and `removed_*` input
- empirical joint resampling
- dependency-aware sprint pulling with whole-item completion
- reporting of sprint-count percentiles, projected dates, recommended planned load, and historical diagnostics

The following items are explicitly **out of MVP scope** and should not block the first implementation increment:

- volatility overlay
- future sprint overrides
- task-level execution spillover and carryover modeling
- burn-up percentile charts
- heterogeneous-task warnings in task mode beyond basic documentation and validation guidance
- external CSV and JSON history-file import

6.1 MVP Step 1. Schema and Validation

Primary files:

- `src/mcprojsim/models/project.py`
- `src/mcprojsim/config.py`
- `src/mcprojsim/parsers/error_reporting.py`
- `src/mcprojsim/parsers/yaml_parser.py`

- `src/mcprojsim/parsers/toml_parser.py`

Work:

1. Add an optional `sprint_planning` field to `Project`.
2. Add `SprintPlanningSpec` and `SprintHistoryEntry` models with MVP fields only.
3. Add any minimum task-level planning field needed for story-point mode, most likely `planning_stor`
4. Implement validation for:

- `sprint_length_weeks`
- `capacity_mode`
- `sprint_id` uniqueness
- exactly one completed-unit field per history row
- unit-family consistency per row and per simulation
- non-negative historical values
- strictly positive `holiday_factor`
- at least two usable historical rows after defaulting

5. Extend source-aware raw validation so malformed sprint-planning files fail with good YAML/TOM line references.

Exit criteria:

- existing project files still parse unchanged;
- minimal and full MVP sprint-planning files validate successfully;
- malformed sprint-planning files fail with precise validation messages.

6.2 MVP Step 2. Historical Normalization and Resampling

Primary files:

- new `src/mcprojsim/planning/sprint_capacity.py`

Work:

1. Implement sparse-row default filling.
2. Implement holiday normalization for delivery-side quantities only.
3. Treat same-cadence historical sprint rows as the primary sampling entity.
4. Implement weekly normalization for mixed sprint lengths as the MVP fallback path, with a clear warning that it is an approximation.
5. Implement joint bootstrap sampling of completed, spillover, added, and removed history.
6. Implement historical diagnostics required by MVP:

- descriptive statistics for each historical series
- spillover, addition, and removal ratios
- Pearson correlation coefficients across the historical series

7. Exclude volatility overlay and future sprint overrides from this MVP sampler.

Exit criteria:

- the sampler produces deterministic outputs for a fixed seed;
- whole-sprint entity sampling works when same-cadence history exists;
- weekly normalization fallback works for mixed sprint lengths;
- the joint sample preserves row-level coupling by construction.

6.3 MVP Step 3. Dependency-Aware Sprint Planner

Primary files:

- new `src/mcprojsim/planning/sprint_planner.py`
- existing `src/mcprojsim/simulation/scheduler.py` as a design reference only

Work:

1. Build a ready queue from existing task dependencies.
2. Apply deterministic ordering using priority then task ID, with task ID order as the fallback.
3. Pull whole tasks into a sprint until sampled capacity would be exceeded.
4. Defer non-fitting tasks without charging capacity.
5. Represent sampled `added` and `removed` units as explicit auditable backlog adjustments rather than as task-execution events.
6. Keep MVP semantics non-preemptive and whole-item only: no task-level execution spillover, no remainder tasks.

Exit criteria:

- whole-item pull works for both capacity modes;
- dependency unlocking works across sprints;
- added and removed work change backlog state explicitly and are traceable through an auditable ledger.

6.4 MVP Step 4. Sprint Simulation Engine and Results Model

Primary files:

- new `src/mcprojsim/planning/sprint_engine.py`
- new `src/mcprojsim/models/sprint_simulation.py`
- existing `src/mcprojsim/simulation/engine.py` for random-seed patterns

Work:

1. Implement an iteration loop that combines the sampler and planner until backlog completion.
2. Record sprint counts and projected completion dates.
3. Compute the planned-load guidance heuristic from the historical series.
4. Add a dedicated `SprintPlanningResults` model with helpers for percentiles and summary statistics.

5. Keep the results surface separate from the existing duration `SimulationResults` model.

Exit criteria:

- the engine produces P50, P80, and P90 sprint counts;
- date projection uses sprint boundaries rather than working-day accumulation;
- the planned-load recommendation is available at the configured planning confidence level.

6.5 MVP Step 5. CLI and Export Integration

Primary files:

- `src/mcprojsim/cli.py`
- `src/mcprojsim/exporters/json_exporter.py`
- `src/mcprojsim/exporters/csv_exporter.py`
- `src/mcprojsim/exporters/html_exporter.py`

Work:

1. Detect enabled sprint planning in `simulate`.
2. Run the existing duration simulation unchanged.
3. Run sprint planning as an additional analysis only when configured.
4. Extend CLI output with MVP sprint-planning summaries.
5. Add dedicated sprint-planning sections to JSON, CSV, and HTML exports.

MVP export content:

- sprint-count percentiles
- projected sprint-based dates
- recommended planned sprint load
- historical completed, spillover, added, and removed statistics
- derived ratio summaries
- Pearson correlation summaries
- selected `removed_work_treatment`

Exit criteria:

- sprint-enabled projects show both simulation views clearly separated;
- non-sprint projects produce unchanged CLI and exporter behavior.

6.6 MVP Step 6. Tests

Primary files:

- `tests/test_models.py`
- `tests/test_parsers.py`
- `tests/test_simulation.py`

- `tests/test_results.py`
- `tests/test_cli.py`
- `tests/test_exporters.py`

Work:

1. Add schema and parser validation tests.
2. Add sampler tests for normalization and joint bootstrap behavior.
3. Add planner tests for deterministic pull order, dependency unlocks, deferral, and backlog adjustments.
4. Add results, CLI, and exporter tests for the MVP output surface.
5. Add at least one end-to-end MVP fixture project.

Exit criteria:

- the MVP sprint-planning path is covered at schema, engine, CLI, exporter, and integration levels;
- existing non-sprint tests remain green.

Chapter 7

Natural-Language Parser Extension Plan

The MCP server already routes natural-language project descriptions through `src/mcprojsim/nl_parser` so sprint-planning natural-language support should be implemented by extending that parser rather than by adding a parallel MCP-only parser. The goal is to parse a semi-structured human description, normalize it into canonical sprint-planning data, and then emit the same YAML structure defined elsewhere in this proposal.

7.1 Main parser functions to change

Primary file:

- `src/mcprojsim/nl_parser.py`

7.1.1 1. Extend the parsed data model

Add sprint-planning dataclasses and attach them to `ParsedProject`.

Recommended additions:

- `ParsedSprintPlanning`
- `ParsedSprintHistoryEntry`
- `ParsedProject.sprint_planning: ParsedSprintPlanning | None`

The parser should keep using lightweight dataclasses in this layer. It should not duplicate the full Pydantic validation model here.

7.1.2 2. Extend `parse()` with sprint-planning sections

`NLProjectParser.parse()` currently tracks task, resource, and calendar sections. It should also track:

- whether the parser is inside a `Sprint planning:` section;

- whether the parser is inside a `Sprint history <id>:` section;
- the current in-progress historical sprint row.

Recommended behavior:

1. Detect a sprint-planning section header before task parsing.
2. Detect repeated sprint-history headers and flush the previous history row when a new one starts.
3. Parse sprint-planning bullets and history bullets using dedicated helper functions rather than overloading task parsing logic.
4. Flush any open sprint-history row before switching into tasks, resources, or calendars.

7.1.3 3. Add sprint-planning metadata helpers

Add focused helpers similar to the existing `_try_parse_project_metadata()` pattern.

Recommended new helpers:

- `_try_parse_sprint_planning_bullet()`
- `_try_parse_sprint_history_bullet()`
- `_try_parse_sprint_length()`
- `_try_parse_capacity_mode()`
- `_try_parse_planning_confidence()`
- `_try_parse_removed_work_treatment()`
- `_try_parse_holiday_factor()`
- `_try_parse_completed_units()`
- `_try_parse_spillover_units()`
- `_try_parse_added_units()`
- `_try_parse_removed_units()`

These helpers should normalize accepted synonyms into canonical internal fields. For example, `done`, `finished`, and `delivered` should map to the completed-work field, while `carryover`, `rolled over`, and `spillover` should all map to the spillover field.

7.1.4 4. Add normalization helpers for units and aliases

The parser will need a small normalization layer so common human expressions map to deterministic YAML.

Recommended helper responsibilities:

- normalize `story points`, `points`, and `pts` to the `story_points` capacity family;
- normalize `tasks` and `items` to the `tasks` capacity family;
- normalize `%` and decimal confidence formats so `80%` and `0.80` both become `0.80`;
- normalize natural-language `holiday_factor` expressions such as `90 percent capacity` into `0.9`;
- normalize removed-work semantics such as `churn only` and `reduce backlog`.

This logic should stay conservative. If the wording is ambiguous, the parser should leave the field unset and let validation fail later with a clear error rather than silently guessing.

7.1.5 5. Extend `to_yaml()` to emit `sprint_planning`

`NLProjectParser.to_yaml()` should emit a canonical `sprint_planning` block when parsed sprint-planning data is present.

At minimum it should output:

- `enabled`
- `sprint_length_weeks`
- `capacity_mode`
- `planning_confidence_level`
- `removed_work_treatment`
- inline `history` rows with canonical field names

This keeps the MCP server contract unchanged: it still returns a normal YAML project file, but that YAML can now include sprint planning.

7.1.6 6. Keep `parse_and_generate()` unchanged in shape

`parse_and_generate()` should remain a thin wrapper over `parse()` and `to_yaml()`. No new MCP-specific branching should be added there. The value comes from extending the parser's canonical understanding of the project description, not from special-casing one caller.

7.2 Suggested implementation order inside `nl_parser.py`

1. Add new dataclasses for sprint planning and history rows.
2. Add regex patterns and alias tables for sprint-planning headers and bullet parsing.
3. Extend `parse()` to recognize sprint-planning and sprint-history sections.
4. Add focused parsing helpers for the new sprint-planning fields.
5. Extend `to_yaml()` to emit the parsed `sprint_planning` block.
6. Add parser tests covering accepted aliases, mixed phrasing, and fallback failures.

7.3 Verification targets for the natural-language path

- A semi-structured MCP description with sprint-planning input generates canonical YAML containing `sprint_planning`.
- Mixed human phrasing such as `done`, `finished`, `carryover`, `rolled over`, and `scope added` normalizes to the canonical history fields.
- Ambiguous free-form text is not silently misparsed as sprint-planning configuration.
- Existing natural-language task, resource, and calendar parsing continues to work unchanged when no sprint-planning section is present.

Chapter 8

Some considerations for using an MCMC model

8.1 The Generative Model

Core Insight

The sprint data has temporal dependencies and unobserved team capacity that varies sprint-to-sprint. A hierarchical Bayesian model with MCMC is perfect here.

Data Structure

For each sprint $t = 1, \ldots, T$, we observe:

- N_t: Team size (known)
- C_t: Stories completed
- S_t: Story points completed
- O_t: Stories spilled over
- A_t: Stories added mid-sprint

Latent Variables (What We're Inferring)

- λ_t: True team velocity (stories/sprint capacity) — varies over time
- ϕ_t: Story complexity (average points per story)
- ρ_t: Scope creep rate (tendency to add work)
- σ_t: Spillover tendency (execution risk)

8.2 The Statistical Model

Level 1: Sprint-Level Likelihoods

Completed Stories:

$$C_t \sim \text{Poisson}(\lambda_t \cdot N_t \cdot \eta_t)$$

Where η_t is sprint efficiency (could be fixed or modeled). The Poisson captures count uncertainty. Story Points (conditional on stories):

$$S_t \mid C_t \sim \text{Gamma}\left(\alpha = C_t \cdot \phi_t, \beta = 1\right)$$

Or equivalently: points per story $\sim$ Exponential$(1/\phi_t)$ Spilled Stories:

$$O_t \sim \text{Binomial}(C_t + O_t, \pi_t)$$

Where $\pi_t = \text{logit}^{-1}(\rho_t)$ is spillover probability. The "denominator" is committed work = completed + spilled. Added Stories:

$$A_t \sim \text{Poisson}(\gamma_t \cdot C_t)$$

Where γ_t is the scope creep ratio relative to planned work.

Level 2: Temporal Dynamics (The Markov Structure)

This is where MCMC shines. Team capability evolves:

$$\lambda_t = \lambda_{t-1} \cdot \epsilon_t^{\delta}, \quad \epsilon_t \sim \text{LogNormal}(0, \sigma_\lambda)$$

Or in log-space (easier for sampling):

$$\log \lambda_t \sim \text{Normal}(\log \lambda_{t-1}, \sigma_\lambda)$$

Same structure for ϕ_t, ρ_t, γ_t — these form a multivariate random walk or vector autoregression to model correlations between capability metrics.

Level 3: Hyperpriors

- $\lambda_0 \sim \text{Gamma}(2, 2/\bar{\lambda}_{\text{empirical}})$
- $\sigma_\lambda \sim \text{HalfNormal}(0.5)$
- $\phi_0 \sim \text{Gamma}(3, 3/\bar{\phi}_{\text{empirical}})$
- $\rho_0 \sim \text{Normal}(0, 1)$ (logit scale)
- $\gamma_0 \sim \text{HalfNormal}(0.3)$

8.3 MCMC Implementation Strategy

Why This Fits MCMC

1. Non-conjugate priors: The logit link for spillover and the random walk structure make Gibbs sampling hard
2. Correlated posteriors: λ_t and λ_{t+1} are strongly coupled — Hamiltonian Monte Carlo (HMC) handles this beautifully
3. Latent variable inference: We never observe "true velocity," only noisy completions

Sampling Approach

Use PyMC or Stan:

```
# Pseudo-code structure
with pm.Model() as sprint_model:
    # Hyperpriors
    sigma_lambda = pm.HalfNormal('sigma_lambda', 0.5)

    # Random walk for log-velocity
    log_lambda = pm.GaussianRandomWalk(
        'log_lambda',
        sigma=sigma_lambda,
        init_dist=pm.Normal.dist(0, 1),
        shape=n_sprints
    )
    lambda_ = pm.Deterministic('lambda', pm.math.exp(log_lambda))

    # Similar for phi, rho, gamma...

    # Likelihoods
    pm.Poisson('completed', mu=lambda_ * team_size * efficiency, observed=completed)

    # Spillover: need to model committed = completed + spilled
    committed = completed + spilled
    pm.Binomial('spilled', n=committed, p=pm.math.invlogit(rho), observed=spilled)

    # Posterior predictive for future sprints...

    trace = pm.sample(2000, tune=1000, target_accept=0.9)
```

8.4 Prediction: Rolling Forward

To predict sprint T+1:

1. Sample $\lambda_{T+1} \sim \text{LogNormal}(\log \lambda_T, \sigma_\lambda)$ from posterior
2. Generate $C_{T+1} \sim \text{Poisson}(\lambda_{T+1} \cdot N_{T+1})$
3. Generate O_{T+1}, A_{T+1}, S_{T+1} from their respective distributions

This gives full predictive distributions, not just point estimates — crucial for sprint planning (e.g., "80% credible interval for completion is 12-18 stories").

8.5 Model Extensions

8.5.1 Extension Implementation

Team changes Add N_t as multiplier, or model onboarding lag Sprint length variation Include duration D_t as offset: $\lambda_t \cdot D_t$ Epic/story type Hierarchical model: $\lambda_t^{(type)}$ with shared hyperpriors Seasonality Add Fourier terms or sprint-of-quarter indicators Overdispersion Replace Poisson with Negative Binomial

8.5.2 Validation Strategy

1. Posterior predictive checks: Simulate historical sprints, compare to actual
2. Leave-future-out validation: Train on sprints $1 \ldots T-k$, predict $T-k+1 \ldots T$, compare to actual
3. Calibration: Are 80% credible intervals actually containing 80% of outcomes?

8.6 Key Advantage of This Approach

Traditional velocity forecasting uses rolling averages — this throws away uncertainty and temporal structure. The MCMC model:

- Propagates uncertainty through the entire prediction
- Adapts to trend changes (improving/declining team)
- Quantifies risk (spillover probability, scope creep)
- Incorporates structural knowledge (team size effects, story point distributions)

Chapter 9

Alternative Approache using Negative Binomial Distribution

If we want a simpler, non-MCMC approach, we could model completed stories with a Negative Binomial to capture overdispersion:

$$C_t \sim \text{NegativeBinomial}(\mu = \lambda_t \cdot N_t, \alpha)$$

Where α captures extra variability beyond Poisson. We could still model λ_t as a random walk to capture temporal dynamics, but we would lose the ability to model spillover and added work explicitly in the likelihood. This might be a reasonable compromise if we want a simpler model with closed-form inference, but it sacrifices some of the richness of the full MCMC approach.

9.1 How to find the model parameters from the historical data?

We can use the historical sprint data to estimate the parameters of our model. For example, we can use maximum likelihood estimation (MLE) or Bayesian inference to fit the parameters λ_t, ϕ_t, ρ_t, and γ_t to the observed data.

In a Bayesian framework, we would specify priors for these parameters and then use MCMC sampling to obtain their posterior distributions given the historical data. This allows us to capture the uncertainty in our parameter estimates and make probabilistic predictions about future sprints.

9.1.1 Using MLE to estimate the parameters

We can set up the likelihood function based on our model and then use optimization techniques to find the parameter values that maximize this likelihood given the historical data. This would give us point estimates for the parameters, but it would not capture the uncertainty in these estimates as a Bayesian approach would. For example, we could use the `scipy.optimize` library in Python to

perform this optimization. However, this approach may be less robust to overfitting and may not provide as rich insights into the uncertainty of our predictions compared to a full Bayesian MCMC approach.

Example code:

```
import numpy as np
from scipy.optimize import minimize
# Assume we have historical data for T sprints
T = 24
N = np.array([5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5])  # Team size
M = np.array([105,68,84,93,102,90,63,67,80,39,66,48,74,87,67,92,87,53,97,94,146,104,111,129])  # Commi
C = np.array([85,63,73,81,92,46,65,31,82,49,86,25,71,71,86,96,55,44,90,46,91,67,46,111])  # Completed
def negative_log_likelihood(params):
    lambda_0, sigma_lambda = params
    # Here we would compute the likelihood of the observed data given the parameters
    # This would involve simulating the random walk for lambda_t and computing the
    # likelihood of C_t given lambda_t and N_t
    # For simplicity, let's assume we have a function that computes this likelihood
    likelihood = compute_likelihood(C, N, lambda_0, sigma_lambda)
    return -likelihood  # We minimize the negative log likelihood
initial_guess = [20, 0.5]  # Initial guess for lambda_0 and sigma_lambda
result = minimize(negative_log_likelihood, initial_guess, bounds=[(0, None), (0, None)])
estimated_lambda_0, estimated_sigma_lambda = result.x
```

9.1.2 Using Bayesian inference to estimate the parameters

In a Bayesian framework, we would specify prior distributions for our parameters based on domain knowledge or previous data. We would then use the observed historical sprint data to update these priors and obtain posterior distributions for the parameters. A full MCMC implementation is still reasonable for the richer state-space model above, but the example below is intentionally lighter weight: it uses a conjugate Gamma-Poisson model over a rolling historical window so it can run as-is with only NumPy. The result is still a probabilistic forecast for the next five sprints, including predictive intervals rather than only point estimates.

Example code:

See: `design-ideas/baysian_example_pred.py` for a full implementation of this approach.

Chapter 10

Addendum: Team-Size-Aware Sprint Forecasting

This addendum extends the sprint-planning design to make explicit statistical use of historical team size per sprint and planned staffing for future sprints.

10.1 Why Team Size Should Be Modeled Explicitly

Historical completed work, spillover, added scope, and removed scope are not only a function of planning quality and churn. They are also influenced by staffing level in that sprint.

If the model ignores team size, it can confound two different effects:

- true process volatility;
- predictable capacity shifts caused by staffing changes.

The practical result is biased forecasts when future staffing differs from historical staffing.

10.2 Statistical Design: Separate Team Capacity From Team Effectiveness

The cleanest model split is:

- **Capacity scale** from team size in a given sprint;
- **Effectiveness profile** from empirical historical outcomes after size normalization.

Let sprint i have:

- n_i: historical team size;
- c_i, s_i, a_i, r_i: completed, spillover, added, removed units;
- h_i: holiday factor;
- L_i: sprint length in weeks.

10.2.1 Step 1: Normalize to per-person baseline

Convert delivery and churn quantities to per-person units (after holiday normalization for delivery-side terms):

$$\tilde{c}_i = \frac{c_i}{\max(n_i, \epsilon) \cdot \max(h_i, \epsilon)}, \quad \tilde{s}_i = \frac{s_i}{\max(n_i, \epsilon) \cdot \max(h_i, \epsilon)}$$

$$\tilde{a}_i = \frac{a_i}{\max(n_i, \epsilon)}, \quad \tilde{r}_i = \frac{r_i}{\max(n_i, \epsilon)}$$

This creates historical observations that are comparable across sprints with different team sizes.

10.2.2 Step 2: Empirical joint resampling on normalized vectors

Use the existing joint empirical approach, but on per-person vectors:

$$\tilde{X}_i = (\tilde{c}_i, \tilde{s}_i, \tilde{a}_i, \tilde{r}_i)$$

Resample these vectors jointly to preserve coupling between delivery and churn.

10.2.3 Step 3: Scale sampled outcomes by future sprint staffing

For future sprint t with planned team size n_t^{future} and override multipliers:

$$c_t^{\text{sampled}} = \tilde{c}_{J_t} \cdot n_t^{\text{future}} \cdot h_t^{\text{override}} \cdot m_t^{\text{override}}$$

$$s_t^{\text{sampled}} = \tilde{s}_{J_t} \cdot n_t^{\text{future}} \cdot h_t^{\text{override}} \cdot m_t^{\text{override}}$$

$$a_t^{\text{sampled}} = \tilde{a}_{J_t} \cdot n_t^{\text{future}}, \quad r_t^{\text{sampled}} = \tilde{r}_{J_t} \cdot n_t^{\text{future}}$$

where:

- h_t^{override} is the future sprint holiday factor (defaults to 1.0);
- m_t^{override} is an explicit capacity multiplier (defaults to 1.0).

This keeps team-size effects explicit and auditable.

10.3 Non-Linear Team-Size Effects (Optional, Recommended)

Linear scaling by headcount is a strong baseline, but real teams often show diminishing returns from coordination overhead.

Add an optional exponent:

$$\text{effective_team_size}(n) = n^{\beta}, \quad 0.75 \leq \beta \leq 1.0$$

Then replace n_t^{future} with $\text{effective_team_size}(n_t^{\text{future}})$ in scaling formulas.

Recommended default for MVP behavior:

- `team_size_scaling_exponent = 1.0` (strict linear scaling).

Recommended post-MVP default once data supports calibration:

- empirical estimate from historical data, clamped into `[0.75, 1.0]`.

10.4 Schema Additions

Add explicit future staffing controls in `sprint_planning`.

10.4.1 New default staffing setting

Add a planning-spec default used when a future sprint has no explicit override:

```
sprint_planning:
  default_future_team_size: 5
```

Behavior:

- if present, this is the baseline team size for all forecasted future sprints;
- if omitted, fallback to last historical `team_size` if available;
- if no historical `team_size` exists either, validation should fail with a clear error requiring one of:
  - `default_future_team_size`, or
  - per-sprint team-size overrides.

10.4.2 Extend `future_sprint_overrides`

Add optional per-sprint staffing override:

```
future_sprint_overrides:
  - sprint_number: 3
    team_size: 4
    holiday_factor: 0.8
    capacity_multiplier: 0.95
    notes: "Two engineers on leave"
```

Interpretation order for each simulated future sprint:

1. resolve base team size from `default_future_team_size`;
2. if matching override has `team_size`, replace base value for that sprint;

3. apply `holiday_factor` and `capacity_multiplier` as separate multiplicative adjustments.

10.5 Updated Canonical Models

Add fields to existing proposed models.

10.5.1 `SprintPlanningSpec` additions

- `default_future_team_size: int | float` (strictly positive)
- `team_size_scaling_exponent: float` in `(0, 1]`, default `1.0`

10.5.2 `FutureSprintOverrideSpec` additions

- `team_size: int | float | None` (strictly positive when present)

10.5.3 `SprintHistoryEntry` expectations

- `team_size` remains optional per row for backward compatibility;
- but team-size-aware mode requires enough historical `team_size` data to calibrate normalization.

10.6 Validation Requirements for Team Size

Add the following validations:

1. All provided historical `team_size` values must be strictly positive.
2. `default_future_team_size`, when provided, must be strictly positive.
3. `future_sprint_overrides[].team_size`, when provided, must be strictly positive.
4. Team-size-aware forecasting requires at least two usable historical rows with positive delivery signal and non-null `team_size`.
5. If team-size-aware forecasting is enabled and future sprint team size cannot be resolved for a simulated sprint, validation fails.
6. `team_size_scaling_exponent` must be in `(0, 1]`.

10.7 Simulation Algorithm Changes

Update the sprint recursion to resolve team size per future sprint and use team-size-normalized empirical samples.

Per iteration, per sprint t:

1. Resolve `team_size_t`:

- from matching override `team_size`, else
- from `default_future_team_size`.

2. Compute effective size:

- $n_t^{\text{eff}} = team_size_t^{\beta}$.

3. Draw one normalized joint vector $(\tilde{c}, \tilde{s}, \tilde{a}, \tilde{r})$.
4. Scale sampled quantities by n_t^{eff} and override multipliers.
5. Pass scaled delivery quantity into dependency-aware planner.
6. Apply scaled added/removed quantities as auditable backlog ledger adjustments.
7. Continue until backlog completion.

This keeps team-size usage explicit in both capacity and churn pathways.

10.8 Reporting and Diagnostics Additions

Include staffing-aware diagnostics in sprint outputs:

- historical per-person completed/spillover/added/removed statistics;
- forecast staffing profile used for each sprint (`team_size`, effective size, overrides applied);
- sensitivity view: completion percentile shifts under staffing scenarios;
- attribution breakdown: how much P80 shift came from staffing vs churn vs holiday factors.

Suggested new summary lines:

- `Historical per-person completed capacity: mean 4.1 SP/person/sprint`
- `Default future team size: 5`
- `Sprint 3 override team size: 4 (holiday_factor 0.8)`
- `P80 completion changed from 7 to 8 sprints due to staffing overrides`

10.9 Design for Future Sprint Staffing Input

Support two ways to specify future staffing:

1. **Global default**

- `default_future_team_size` at `sprint_planning` level.

2. **Per-sprint override**

- `future_sprint_overrides[].team_size`.

This exactly matches typical planning workflows:

- expected steady-state team size as default;
- explicit exceptions for known leave, onboarding, or temporary staffing changes.

10.10 Example: Complete Team-Size-Aware Sprint Planning Block

```yaml
sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  planning_confidence_level: 0.80
  removed_work_treatment: churn_only
  default_future_team_size: 6
  team_size_scaling_exponent: 0.92
  history:
    - sprint_id: SPR-001
      sprint_length_weeks: 2
      team_size: 5
      completed_story_points: 23
      spillover_story_points: 4
      added_story_points: 3
      removed_story_points: 1
    - sprint_id: SPR-002
      sprint_length_weeks: 2
      team_size: 6
      completed_story_points: 25
      spillover_story_points: 6
      added_story_points: 4
      removed_story_points: 2
  future_sprint_overrides:
    - sprint_number: 2
      team_size: 4
      holiday_factor: 0.8
      notes: "Conference week"
    - sprint_number: 4
      team_size: 7
      capacity_multiplier: 1.05
      notes: "Temporary staffing uplift"
```

10.11 Implementation Plan in This Repository

10.11.1 1. Schema layer

Update:

- `src/mcprojsim/models/project.py`

Changes:

- add `default_future_team_size` and `team_size_scaling_exponent` to `SprintPlanningSpec`;
- add `team_size` to `FutureSprintOverrideSpec`;
- validate positivity and bounds.

10.11.2 2. Parser and validation layer

Update:

- `src/mcprojsim/parsers/error_reporting.py`
- `src/mcprojsim/parsers/yaml_parser.py`
- `src/mcprojsim/parsers/toml_parser.py`

Changes:

- add source-aware errors for invalid team-size fields;
- ensure future sprint team size is resolvable for simulation horizon.

10.11.3 3. Sprint sampler and engine layer

Update:

- `src/mcprojsim/planning/sprint_capacity.py`
- `src/mcprojsim/planning/sprint_engine.py`
- `src/mcprojsim/planning/sprint_planner.py`

Changes:

- normalize historical data to per-person vectors;
- sample joint normalized vectors;
- scale by future effective team size and overrides;
- preserve auditable backlog ledger.

10.11.4 4. Results and exporter layer

Update:

- `src/mcprojsim/models/sprint_simulation.py`
- `src/mcprojsim/exporters/json_exporter.py`
- `src/mcprojsim/exporters/csv_exporter.py`
- `src/mcprojsim/exporters/html_exporter.py`

Changes:

- add staffing profile and per-person historical diagnostics;
- export override attribution and scenario deltas.

10.11.5 5. CLI and MCP layer

Update:

- `src/mcprojsim/cli.py`
- `src/mcprojsim/mcp_server.py`
- `src/mcprojsim/nl_parser.py`

Changes:

- allow setting default future team size from CLI/MCP/NL flows;
- allow setting per-sprint override team size in structured YAML and NL parser aliases.

10.12 Backward Compatibility and Migration

To avoid breaking existing sprint-planning project files:

1. Keep `team_size` optional in historical rows.
2. Keep team-size-aware scaling disabled unless sufficient team-size data exists.
3. If team-size-aware mode is disabled, keep current empirical behavior unchanged.
4. Provide a clear warning when team-size data is partial and the model falls back.

Recommended explicit mode flag:

```
sprint_planning:
  team_size_mode: auto   # auto | enabled | disabled
```

Behavior:

- `disabled`: ignore team size completely.
- `enabled`: require complete necessary inputs and fail if missing.
- `auto` (default): use team-size-aware model when inputs are sufficient, otherwise fallback with warning.

10.13 Best-Use Recommendation

For the best statistical use of this new data in production forecasting:

1. Use joint empirical resampling on per-person normalized vectors.
2. Scale by explicit future staffing (`default_future_team_size` + override `team_size`).
3. Keep holiday and capacity multipliers separate from staffing.
4. Add optional non-linear scaling via `team_size_scaling_exponent`.
5. Report staffing attribution so users can see when forecast shifts are staffing-driven versus churn-driven.

This design preserves the existing Monte Carlo philosophy, materially improves forecast realism under staffing changes, and keeps implementation aligned with current `mcprojsim` architecture.